

基于几何解析与数值模拟的无人机烟幕干扰投放策略

摘要

本文围绕导弹防御中烟幕干扰弹（下文简称烟幕弹）协同遮蔽策略优化问题，基于**运动学建模、几何解析与优化算法**，系统研究了从单一无人机到多无人机、从单个烟幕弹到多个烟幕弹及从单个导弹到多个导弹等不同任务场景下的最优决策方案。本文全部数学模型的计算与模拟过程均**通过 MATLAB 编程实现**，确保了算法的可执行性与结果的可复现性。

针对问题一，本文基于给定参数建立了导弹、无人机与烟幕弹的运动学模型，通过解析计算确定烟幕弹的投放与起爆点，并数值模拟云团下沉过程。本文提出**几何解析法**判断真目标是否被有效遮蔽，即通过分析烟幕云团与目标上下底面圆周之间的几何关系，**构造四次多项式并以该式全局非负作为有效遮蔽的代数判据**。采用 $\Delta t = 0.005s$ 步长采样，逐帧判断遮蔽状态，最终得到**有效遮蔽时长为 $1.4050s$** ，**临界时刻示意图见图9**。

针对问题二，以有效遮蔽时长最大化为目标，构建了以无人机航向角、速度、烟幕弹投放时刻和起爆延时为决策变量的**非线性优化模型**。采用**遗传算法 (GA)** 进行全局优化，通过迭代评估获得最优策略，得到**有效遮蔽时长为 $4.5000s$** ，**决策变量组合见表3**。

针对问题三，在问题二基础上将烟幕弹数量增至三个，决策变量扩展为无人机航向角、飞行速度及各弹投放时间与起爆延时。因**几何解析法失效（如图10）**，改用采样法判断有效遮蔽。经遗传算法优化，得到**最优航向角 $\theta_1 = 179.5595^\circ$** 、**飞行速度 $v_1 = 121.2417m/s$** ，**各烟幕弹参数见表5**，**联合遮蔽时长达 $6.8800s$** 。**单个烟幕弹提供的有效遮蔽时长见表8**。对单个烟幕弹提供的有效遮蔽时长进行解析法与采样法**交叉验证**，结果一致，证明了结果可靠性。

针对问题四，将模型扩展为**多无人机-单烟幕弹模型**，每架无人机的航向、飞行速度，每个烟幕弹的投放时间和起爆延时作为决策变量。以总遮蔽时长最大化为目标，**为避免无效搜索以问题二最优解为初始值**。求解得到**联合遮蔽时长为 $10.6500s$** ，**各无人机参数见表8**，**各弹参数与单独遮蔽时长见表9**，结果经交叉验证一致。

针对问题五，在问题三和四的建模基础上，进一步构建了面向**多无人机-多烟幕弹-多导弹模型**，共包含 40 个决策变量。以最大化三枚导弹的总有效遮蔽时长为目标，采用遗传算法进行全局优化求解；为提升遗传算法收敛效率与稳定性，**将问题三的单无人机优化结果作为初始解**。求解得到最优投放策略下**总遮蔽时长达到 $31.0800s$** ，正文中展示了无人机 FY1 相关的决策参数见表11。

关键字： 烟幕弹投放策略 非线性优化 遗传算法 数值模拟 几何解析

一、 问题分析

1.1 问题一的分析

问题一要求利用无人机 FY1 投放一枚烟幕弹对导弹 M1 进行干扰，并在给定无人机飞行速度、投放时间、起爆延时等条件下，计算烟幕对真目标的有效遮蔽时长。建立以假目标为原点的空间直角坐标系，在该坐标系下，计算导弹和无人机的运动轨迹，确定烟幕弹投放点与起爆点位置，并模拟烟幕云团形成后的下沉过程。本文基于有效遮蔽的几何关系，**将圆柱形真目标的有效遮蔽问题转化为上下底面圆周的有效遮蔽判断，进而根据几何关系构造四次多项式并验证其全局非负性，将几何条件转化为代数形式的充分必要条件。**最后，通过在烟幕有效期内以高精度时间步长进行离散采样，逐时刻判断遮蔽状态，确定了遮蔽开始和结束的临界时刻，计算时间差即为有效遮蔽时长。

1.2 问题二的分析

问题二要求在仅使用无人机 FY1 投放一枚烟幕弹的情况下，通过优化无人机的飞行方向、飞行速度、烟幕弹投放时刻和起爆延时，使得对真目标的遮蔽时间尽可能长。本文在问题一模型的基础上，将这四个参数作为决策变量，构建了一个以总遮蔽时长为目标函数、以变量取值范围和物理可行性为约束条件的非线性优化模型。由于目标函数具有**高度非线性、多峰性、无梯度信息**的特性，本文选用**遗传算法（GA）**进行全局优化。经过迭代计算，最终得到一组最优参数组合，以及该策略下的有效遮蔽时长。

1.3 问题三的分析

问题三进一步考虑无人机 FY1 连续投放三枚烟幕弹以延长对真目标的有效遮蔽时长。由于多枚烟幕弹形成的云团在空间和时间上可能存在重叠或间隔，**问题一、二中基于单一烟幕的几何判定方法不再适用。**因此，本文使用**采样法**进行有效遮蔽判断。在此基础上，以三枚烟幕弹的投放时刻和起爆延时为决策变量，构建以总遮蔽时长最大化为目标的非线性优化模型，并采用**遗传算法**进行求解，最终得到最优投放策略及对应的遮蔽时长。将得到的最优策略参数带入问题一的求解过程得到每个烟幕弹单独作用的有效遮蔽时长，并采用**问题一中的解析方法与问题三的采样法进行交叉验证。**结果表明两种方法所得遮蔽时长完全一致，确保结果不会因采样点数量不足而不准确。

1.4 问题四的分析

问题四要求在多架无人机各投放一枚烟幕弹的情况下，通过优化各无人机的航向角、飞行速度、烟幕弹投放时刻与起爆延时，实现对真目标遮蔽时间的最大化。本文在问题二模型的基础上，**将单无人机-单烟幕弹模型扩展为多无人机-单烟幕弹模型**。每架无人机的运动学模型与烟幕弹的投放、起爆、云团下沉过程均独立建模。为避免随机初始化可能导致收敛缓慢或陷入局部最优的问题，首先将每架无人机仅投放一枚烟幕弹的情形代入问题二的模型中求解，并以此结果作为本问题求解策略参数的初始值。其他模型求解流程与问题三的求解流程保持一致。

1.5 问题五的分析

问题五需在五架无人机各投三枚烟幕弹的场景下，优化无人机航向、速度、投弹时机及起爆延时，以最大化三枚导弹对目标的总遮蔽时间。本文建立**多无人机-多烟幕弹-多导弹模型**，同样采用遗传算法进行全局优化。由于决策变量多达 40 个，目标函数复杂且非线性，算法以前期单无人机优化结果为初始值，提升搜索效率。遮蔽判断沿用采样法，为确保结果准确可靠，最终通过交叉验证保证结果准确性。

二、 模型假设

1. 来袭导弹视作匀速直线飞行，速度恒定，导弹飞行方向指向假目标。
2. 无人机视作质点，在受令时瞬时调整航向，随后以固定速度沿固定方向直线匀速飞行。
3. 忽略运动中物体所受的所有阻力。
4. 烟幕弹被瞬时投放，且其在竖直方向上做自由落体运动，起爆前水平速度保持与无人机一致。
5. 烟幕弹起爆瞬时即时形成球形烟幕云团，视作理想球体，且其半径在有效期内变化可忽略。
6. 烟幕弹起爆后无水平速度，仅在竖直方向上做匀速直线运动。
7. 忽略导弹在遇到云团后改变轨迹或被动躲避的可能性，且来袭导弹具有全向探测能力，视野范围内目标始终可被探测。

三、符号说明

表 1 符号说明表

符号	含义	单位
M_i	第 i 枚来袭导弹被警戒雷达发现时的坐标, $i \in \{1, 2, 3\}$	m
A_j	第 j 架无人机位置的坐标, $j \in \{1, 2, 3, 4, 5\}$	m
$M_i(t)$	导弹 M_i 在时间 t 的位置向量	m
$A_j(t)$	第 j 架无人机在时间 t 的位置向量	m
v_m	导弹飞行的速度大小	m/s
$\vec{m}_i$	导弹 M_i 飞行方向的单位方向向量	m
v_j	第 j 架无人机飞行的速度大小	m/s
$\vec{a}_j$	第 j 架无人机飞行方向的单位方向向量	m
$t_{j,k}^d$	第 j 架无人机投放第 k 枚烟幕弹的时间, $k \in \{1, 2, 3\}$	s
$P_{j,k}^d$	第 j 架无人机投放第 k 枚烟幕弹的坐标	m
$t_{j,k}^b$	第 j 架无人机投放第 k 枚烟幕弹引爆的时间	s
$P_{j,k}^b$	第 j 架无人机投放第 k 枚烟幕弹引爆的坐标	m
$C_{j,k}(t)$	第 j 架无人机投放第 k 枚烟幕弹引爆后 球状烟幕云团球心坐标	m
θ_j	第 j 架无人机飞行的航向角, 以 x 轴为正向, 逆时针方向为正, $\theta \in [0, 2\pi]$	rad
$\tau_{j,k}$	第 j 架无人机投放的第 k 枚烟幕弹 延时起爆时间	s
v_s	云团下降速度大小	m/s
R_s	云团有效遮蔽半径	m
T_e	云团有效遮蔽时长	s
r	真目标圆柱底面半径	m
h	真目标圆柱高度	m
g	重力加速度, 取 $9.81m/s^2$	m/s^2

四、问题一的求解

4.1 模型构建

4.1.1 空间直角坐标系的建立

在建立导弹、无人机、烟幕弹的运动模型时，根据题目描述，如图1所示，本文建立如下空间直角坐标系：以假目标为坐标原点 $O(0,0,0)$ ，水平面为 xOy 平面， z 轴垂直向上。本文的所有运动与位置关系均在此坐标系下进行分析。

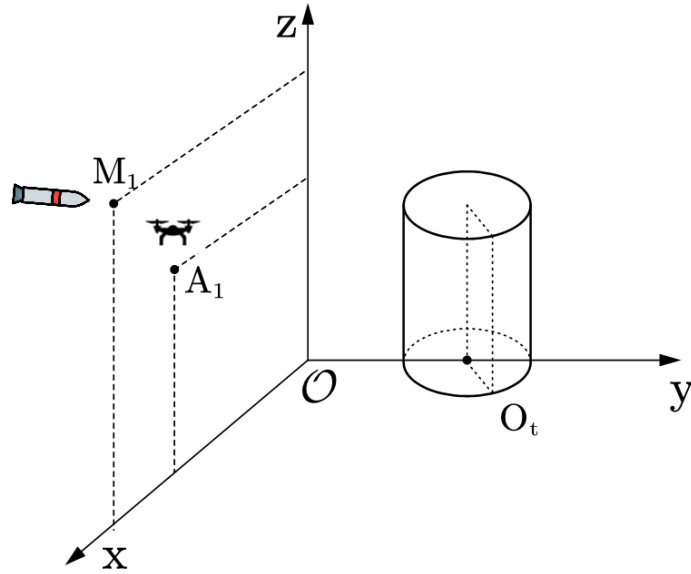


图 1 空间直角坐标系示意图

其中， M_1 表示导弹初始位置， A_1 表示无人机初始位置， O_t 表示圆柱形真目标底面圆心坐标。为清晰展示空间关系，图中各要素的相对尺寸及距离未按实际比例绘制。

4.1.2 导弹轨迹的计算

导弹 M_1 被观测到时的初始位置为 $\mathbf{M}_1$ ，假目标位于坐标原点。 M_1 以恒定速度 v_m 匀速飞向假目标。其单位航向向量为

$$\vec{\mathbf{m}}_1 = \frac{\mathbf{0} - \mathbf{M}_1}{\|\mathbf{0} - \mathbf{M}_1\|}. \quad (1)$$

则 M_1 在 t 时刻的位置为

$$\mathbf{M}_1(t) = \mathbf{M}_1 + v_m t \vec{\mathbf{m}}_1. \quad (2)$$

4.1.3 烟幕弹遮蔽范围的计算

本文对烟幕弹遮蔽范围的计算分为以下几个部分：计算无人机 FY1 运动轨迹，根据题目提供的投放烟幕弹时间计算出烟幕弹的投放点坐标；计算烟幕弹落体运动轨迹，根据题目提供的烟幕弹引爆时间计算出烟幕弹起爆点的坐标；计算烟幕云团中心下沉的运动轨迹，遮蔽范围为以云团中心为球心，半径为 $10m$ 的球体。

Step.1: 无人机 FY1 轨迹与投放点坐标的计算

由题意，无人机 FY1 以固定速度 v_1 等高度飞行，故无人机 FY1 飞行方向应平行于平面 xOy 。设无人机 FY1 航线与 x 轴夹角为 θ_1 (如图2所示)，则无人机单位航向向量为

$$\vec{a}_1 = (\cos \theta_1, \sin \theta_1, 0) \quad (3)$$

问题一中给定了无人机的飞行目标 $\mathbf{A}_0$ ，故此时无人机飞行方向可由下式所得

$$\vec{a}_1 = \frac{\mathbf{A}_0 - \mathbf{A}_1}{\|\mathbf{A}_0 - \mathbf{A}_1\|}. \quad (4)$$

其中， $\mathbf{A}_0$ 为无人机 FY1 航向终点，其与无人机 FY1 初始位置等高。

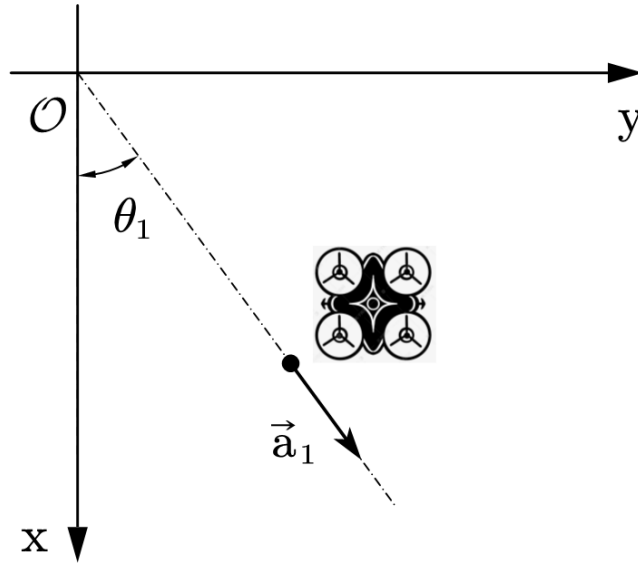


图2 无人机飞行方向俯视图

无人机 FY1 在受令瞬间开始计时，此时 $t = 0s$ 。则无人机在 t 时刻的位置为

$$\mathbf{A}_1(t) = \mathbf{A}_1 + v_1 t \vec{a}_1. \quad (5)$$

当时间为 $t_{1,1}^d$ 时投放烟幕弹，投放点坐标为

$$\mathbf{p}_{1,1}^d = \mathbf{A}_1(t_{1,1}^d). \quad (6)$$

Step.2: 弹体落体轨迹与起爆点坐标的计算

烟幕弹在释放后，仅在重力作用下做水平速度为 v_1 的平抛运动。注意到问题一中无人机 FY1 初始位置的 y 坐标为 0，因此无人机 FY1 的运动轨迹与弹体落体的运动轨迹均在平面 xOz 中，如图3所示。

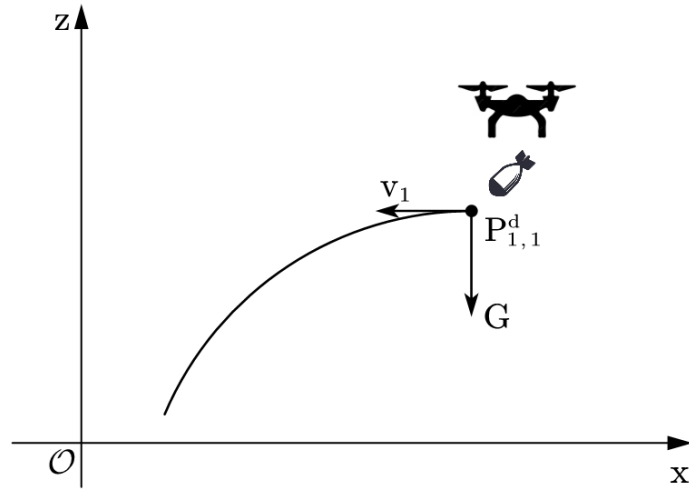


图3 烟幕弹运动轨迹示意图

分别令弹体水平速度在 x 轴方向上和 y 轴方向上的分量为 $v_{h,x}$ 和 $v_{h,y}$ ，则 $t \in [t_{1,1}^d, t_{1,1}^b]$ 时，弹体位置的 $x(t)$ 、 $y(t)$ 、 $z(t)$ 坐标分别为

$$x(t) = x_{1,1}^d + v_{h,x}(t - t_{1,1}^d), \quad (7)$$

$$y(t) = y_{1,1}^d + v_{h,y}(t - t_{1,1}^d), \quad (8)$$

$$z(t) = z_{1,1}^d - \frac{1}{2}g(t - t_{1,1}^d)^2. \quad (9)$$

在延时 $\tau_{1,1}$ 后起爆，则起爆点坐标为

$$\mathbf{p}_{1,1}^b = (x_{1,1}^d + v_{h,x}\tau_{1,1}, y_{1,1}^d + v_{h,y}\tau_{1,1}, z_{1,1}^d - \frac{1}{2}g\tau_{1,1}^2), \quad (10)$$

其中

$$\tau_{1,1} = t_{1,1}^b - t_{1,1}^d = 3.6 \text{ s}. \quad (11)$$

Step.3: 云团中心下沉轨迹的计算

烟幕弹在起爆瞬间形成球状云团，云团沿 z 轴负方向做速度为 v_s 的匀速运动。则 $t \in [t_{1,1}^b, t_{1,1}^b + T_e]$ 时，云团中心坐标关于时间的表达式为

$$\mathbf{C}_{1,1}(t) = \mathbf{p}_{1,1}^b - (0, 0, v_s(t - t_{1,1}^b)). \quad (12)$$

4.1.4 有效遮蔽的定义

为了判断烟幕弹生成的球状云团是否能够有效干扰导弹对真实目标的识别，需要建立针对有效遮蔽的严格数学判定准则。根据题目描述，本文给出的有效遮蔽的定义如下。

定义 1 (有效遮蔽) 在时刻 t ，若点集 $\mathcal{T}$ 上任意一点 P 与导弹位置 $M_1(t)$ 的连线 $M_1(t)P$ 上存在一点 Q ， Q 在某个球状烟雾云团（设为球体 $S(t)$ ）内，如图4所示。即满足：

$$\forall P \in \mathcal{T}, \exists Q \in S(t), \text{ s.t. } Q \in M_1(t)P$$

则称点集 $\mathcal{T}$ 被有效遮蔽。其中，球状烟幕云团 $S(t)$ 是以起爆后中心点 $C(t)$ 为球心、半径 $R_s = 10 \text{ m}$ 的球体，且中心点以 3 m/s 的速度匀速下沉。

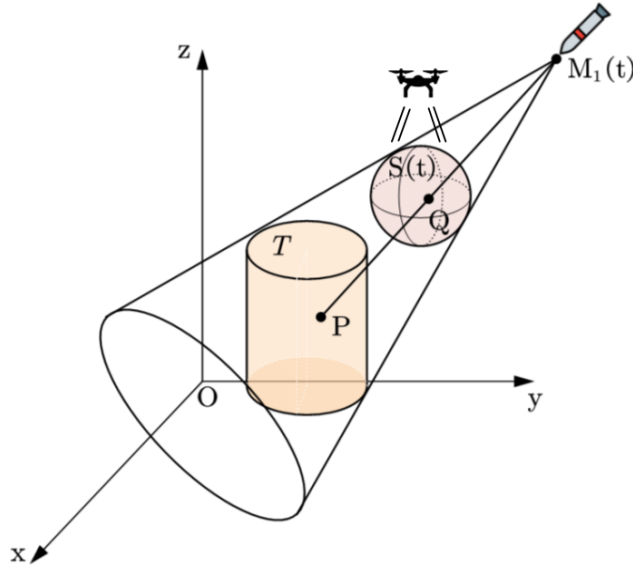


图4 有效遮蔽定义

4.1.5 有效遮蔽的几何判定

几何判定结论的证明 当圆柱形真目标被球状烟幕云团有效遮蔽时（如图5所示）。针对有效遮蔽的几何判定，本文结论为：若圆柱形的上下底面的圆周均被有效遮蔽，则整个

圆柱形真目标被单个烟幕云团有效遮蔽。

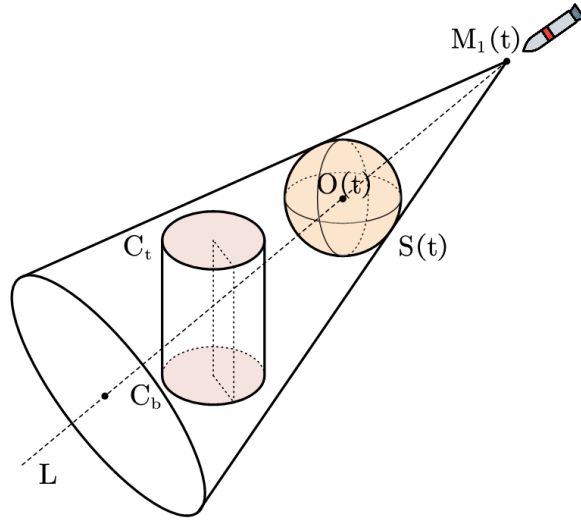


图5 有效遮蔽示意图

设 t 时刻的烟雾云团为球体 $S(t)$ ，其球心位于点 $O(t)$ ，半径为 R_s 。则以 t 时刻导弹位置 $M_1(t)$ 为顶点、与球体 $S(t)$ 相切的所有射线，共同构成了一个圆锥面。该圆锥面即为 t 时刻导弹的监测视野范围，其母线与球体 $S(t)$ 相切。如图5， L 表示圆锥轴线， C_t 表示上底面圆周， C_b 表示下底面圆周。若点集 $C_t \cup C_b$ 满足定义1，则整个圆柱形真目标被单个烟幕云团有效遮蔽。

为了证明上述结论，本文引入引理如下：

引理 1 对于任意线段，若线段的两个端点被有效遮蔽，则该线段上任意一点均被有效遮蔽。即对于线段 AB ，若满足

$$\exists A', B' \in S, s.t. A' \in AM, B' \in BM$$

则

$$\forall P \in AB, \exists P' \in S, s.t. P' \in PM$$

证明 1 如图6所示，任取 $P \in AB$ ，连接 $A'B'$ ， PM 交于点 P' ，因为 $A', B' \in S$ 且 $P' \in A'B'$ ，所以 $P' \in S$ ，由定义1，点 P 被单个烟幕云团 S 有效遮蔽。

考虑整个圆柱形真目标，如图7a所示，当点集 $C_t \cup C_b$ 满足定义1时，对上下底面圆周 C_t 和 C_b 的任意直径 d 应用引理1，得到上下底面圆周内任意一点被有效覆盖。对任意包含于圆柱形内且与地面垂直的线段 h 应用引理1，得到整个圆柱体点集满足定义1，即整个圆柱形真目标被单个烟幕云团有效覆盖。

至此本文完成了对有效遮蔽的几何判定的结论证明。

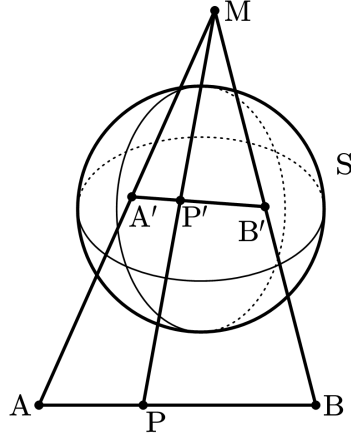
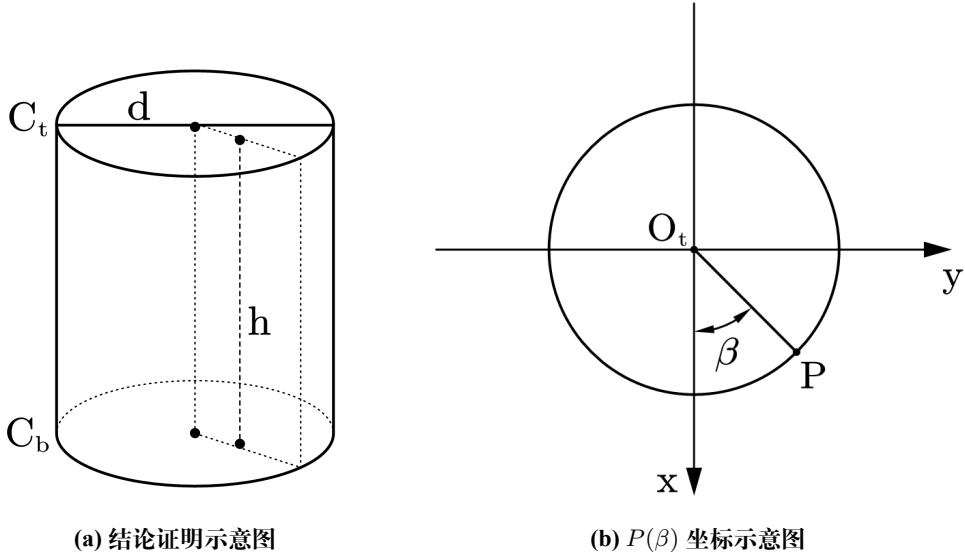


图 6 引理证明示意图



(a) 结论证明示意图

(b) $P(\beta)$ 坐标示意图

图 7

几何判定结论的应用 以下论证均默认在 t 时刻，即对于 t 时刻如何判断圆周是否被有效遮蔽。

以圆柱形上底面圆周 C_t 为例，如图7b所示，设圆周圆心坐标为 O_t ， P 为圆周上一点， β 为 x 轴方向与 O_tP 的夹角，逆时针方向为正， $\beta \in [0, 2\pi)$ 。易得 P 点坐标 $P(\beta)$ 为：

$$P(\beta) = C_t + r(\cos \beta, \sin \beta, 0) \quad (13)$$

当圆柱形上底面圆周 C_t 被有效遮蔽时（如图8所示），由几何关系易知 $P(\beta)M$ 与导弹视野圆锥轴线 L 的夹角 γ 小于导弹视野圆锥的半顶角 α 。若 $\gamma \geq \frac{\pi}{2}$ ，则点 $P(\beta)$ 一定不被有效遮蔽。当 $\gamma \leq \frac{\pi}{2}$ 时， $\gamma < \alpha$ 等价于 $\cos \gamma > \cos \alpha$ 。由此得到**圆周被有效遮蔽的**

充分必要条件为：

$$g(\beta) = \cos \gamma(\beta) - \cos \alpha \geq 0, \quad \forall \beta \in [0, 2\pi) \quad (14)$$

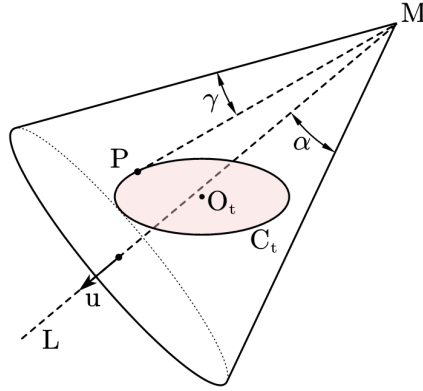


图 8 $P(\beta)$ 坐标示意图

等价变换：

根据向量夹角公式，有：

$$\cos \gamma(\beta) = \frac{(P(\beta) - M) \cdot \mathbf{u}}{\|P(\beta) - M\|}$$

代入式 (14) 得：

$$g(\beta) = \frac{(P(\beta) - M) \cdot \mathbf{u}}{\|P(\beta) - M\|} - \cos \alpha \geq 0 \quad (15)$$

其中 $\mathbf{u}$ 为导弹视野圆锥轴方向单位向量。

将式 (15) 写为分式形式：

$$g(\beta) = \frac{N(\beta)}{D(\beta)}$$

其中：

$$N(\beta) = (P(\beta) - M) \cdot \mathbf{u} - \cos \alpha \cdot \|P(\beta) - M\|$$

$$D(\beta) = \|P(\beta) - M\|$$

由于 $D(\beta) > 0$ 恒成立， $g(\beta)$ 的符号完全由分子 $N(\beta)$ 决定，故原问题等价于：

$$N(\beta) \geq 0, \quad \forall \beta \in [0, 2\pi) \quad (16)$$

前提条件：

令 $A(\beta) = (P(\beta) - M) \cdot \mathbf{u}$, $B(\beta) = \cos \alpha \cdot \|P(\beta) - M\|$, 则:

$$N(\beta) = A(\beta) - B(\beta)$$

注意到 $B(\beta) > 0$ 恒成立。若 $A(\beta) < 0$, 则 $N(\beta) < 0$ 恒成立, 即该点不被有效遮蔽。因此, 必须首先确保:

$$A(\beta) = (P(\beta) - M) \cdot \mathbf{u} \geq 0, \quad \forall \theta \in [0, 2\pi) \quad (17)$$

这一条件可通过计算圆周上 $A(\theta)$ 的最小值进行验证:

$$A_{\min} = s_0 - r\sqrt{A^2 + B^2}$$

其中 $s_0 = (C_t - M) \cdot \mathbf{u}$, $A = u_x$, $B = u_y$ 。若 $A_{\min} < 0$, 则可立即判定圆周不被有效遮蔽。

最终判定形式:

在满足前提条件 (17) 的情况下, $A(\beta) \geq 0$ 且 $B(\beta) > 0$ 。此时不等式 $N(\beta) \geq 0$ 等价于:

$$A(\beta) \geq B(\beta)$$

由于两边均为非负数, 平方操作保持不等号方向:

$$[A(\beta)]^2 \geq [B(\beta)]^2$$

展开即得:

$$[(P(\beta) - M) \cdot \mathbf{u}]^2 \geq \cos^2 \alpha \cdot \|P(\beta) - M\|^2$$

定义函数:

$$h(\beta) = [(P(\beta) - M) \cdot \mathbf{u}]^2 - \cos^2 \alpha \cdot \|P(\beta) - M\|^2 \quad (18)$$

则圆周 C_t 被有效遮蔽的充分必要条件为:

$$h(\beta) \geq 0, \quad \forall \beta \in [0, 2\pi)$$

将 $P(\beta) = C_t + r(\cos \beta, \sin \beta, 0)$ 代入 $h(\beta)$ 并展开, 可得:

$$h(\beta) = C_0 + C_1 \cos \beta + C_2 \sin \beta + C_3 \cos 2\beta + C_4 \sin 2\beta$$

其中系数为：

$$\begin{aligned}
C_0 &= s_0^2 - k(W_2 + r^2) + \frac{1}{2}r^2(A^2 + B^2) \\
C_1 &= 2r(s_0A - kC_1) \\
C_2 &= 2r(s_0B - kC_2) \\
C_3 &= \frac{1}{2}r^2(A^2 - B^2) \\
C_4 &= r^2AB
\end{aligned}$$

这里 $k = \cos^2 \alpha$, $W_2 = \|C_t - M\|^2$, $C_1 = (C_t - M)_x$, $C_2 = (C_t - M)_y$ 。
通过变量代换 $m = \tan \frac{\beta}{2}$, 将 $h(\beta)$ 转化为四次多项式：

$$P(m) = c_4m^4 + c_3m^3 + c_2m^2 + c_1m + c_0 \quad (19)$$

其中系数为：

$$\begin{aligned}
c_0 &= C_0 + C_1 + C_3 \\
c_1 &= 2C_2 + 4C_4 \\
c_2 &= 2C_0 - 6C_3 \\
c_3 &= 2C_2 - 4C_4 \\
c_4 &= C_0 - C_1 + C_3
\end{aligned}$$

至此，本文将圆周是否被有效遮蔽问题转化为判断四次多项式 $P(m)$ 在实数域上的是否全局非负问题。

本文算法中判断多项式 $P(m)$ 在实数域上的是否全局非负分为以下四步：

1. 检查首项系数 c_4 ：若 $c_4 < -\varepsilon$ ($\varepsilon = 10^{-12}$)，则 $P(m) \rightarrow -\infty$ (当 $m \rightarrow \pm\infty$)，直接返回 false
2. 处理降阶情况：若 $|c_4| < \varepsilon$ ，多项式可能降阶，需在 $m = \pm 10^6$ 处采样检查
3. 寻找极值点：计算导数 $P'(m)$ 的实根，并加入候选点列表，同时包含 $m = 0$ (对应 $\beta = 0$)
4. 采样检查最小值：计算所有候选点处的函数值，若最小值 $P_{\min} < -\delta$ ($\delta = 10^{-9}$)，则返回 false；否则返回 true

详细代码见附录问题一代码。

4.2 模型求解

4.2.1 有效遮蔽时长算法

注意到问题一中导弹 M1 和无人机 FY1 的初始位置 y 坐标均为 0，且都向原点方向飞行，因此问题一中导弹 M1 与无人机 FY1 的运动过程在平面 xOy 中，如图9所示。由导弹 M1 和烟幕弹 S 的运动轨迹知，真目标被有效遮蔽的时间是连续的。因此存在时刻 t_1 使得真目标恰好被有效遮蔽，存在时刻 t_3 使得真目标恰好结束被有效遮蔽，时刻 t_2 为导弹进入烟幕云团范围的时刻，当 $t \in [t_2, t_3]$ 时，导弹在烟幕云团内部，真目标一定被有效遮蔽。因此，真目标被有效遮蔽的总时长为 $t_3 - t_1$ 。

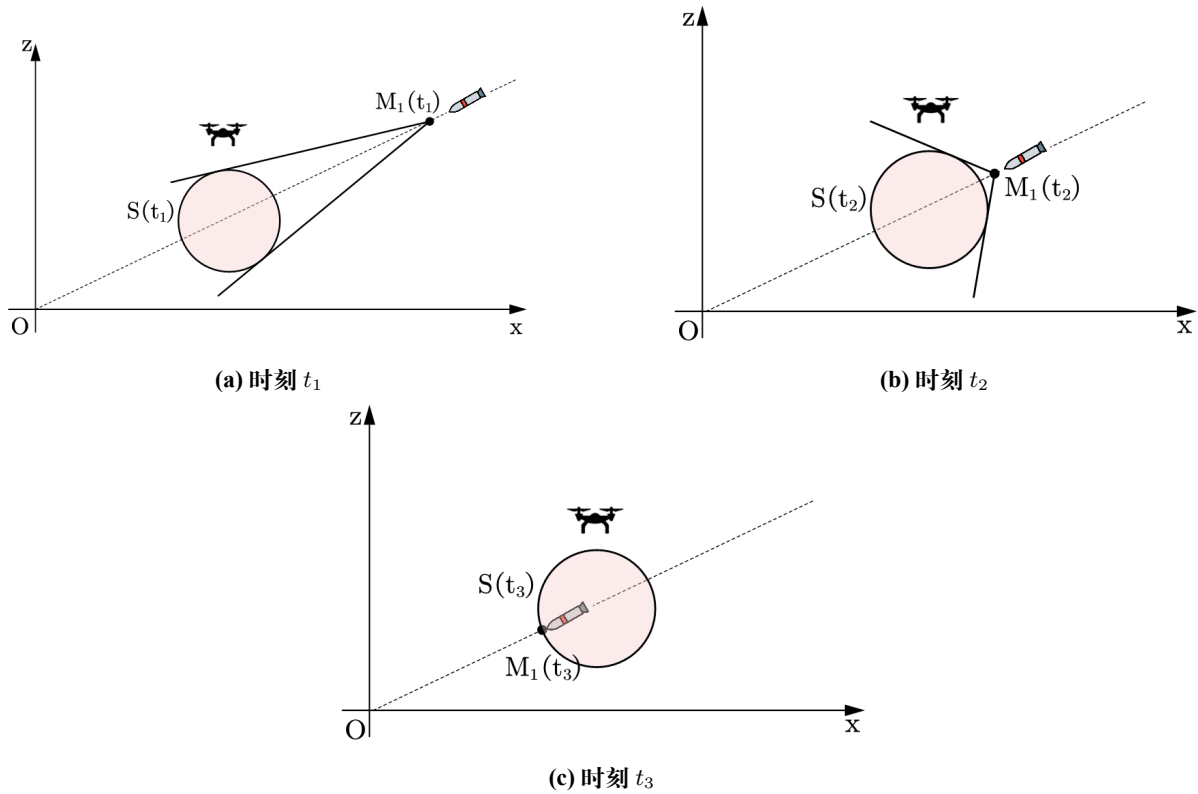


图9 有效遮蔽临界情况示意图

4.2.2 数值模拟流程

本文使用 MATLAB R2022b 对构建的模型进行数值模拟。初始化导弹 M1，无人机 FY1 初始位置，导弹速度、无人机速度、重力加速度、烟幕云团下沉速度、烟幕有效半径。模拟时间范围为从烟幕弹起爆时刻开始，覆盖整个有效遮蔽期 ($T_e = 20s$)，即 $t \in [t_{1,1}^b, t_{1,1}^b + T_e] = [5.1, 25.1]s$ 。模拟总时长为 $T_e = 20s$ ，时间步长设置为 $\Delta t = 0.005s$ ，总模拟步数为 $N = 4000$ 步。

对于每一个时刻 $t_{1,1}^b + k\Delta t$ (k 为用于离散时间序列的整数索引变量)，若该时刻由圆柱形上下底面圆周转化而来的四次多项式(19)都在实数域上非负，则该时刻真目标

被有效遮蔽，否则该时刻真目标不被有效遮蔽。若时刻 $t_{1,1}^b + k\Delta t$ 真目标不被有效遮蔽且时刻 $t_{1,1}^b + (k+1)\Delta t$ 真目标被有效遮蔽，则近似认为真目标恰好被有效遮蔽的时刻 $t_1 = t_{1,1}^b + \frac{2k+1}{2}\Delta t$ 。同理，若时刻 $t_{1,1}^b + h\Delta t$ (h 为用于离散时间序列的整数索引变量) 真目标被有效遮蔽且时刻 $t_{1,1}^b + (h+1)\Delta t$ 真目标不被有效遮蔽，则近似认为真目标恰好不被有效遮蔽的时刻 $t_3 = t_{1,1}^b + \frac{2h+1}{2}\Delta t$ 。

设计算 t_1 和 t_3 的误差分别为 ϵ_1 和 ϵ_3 ，有 $0 < |\epsilon_1| < \Delta t, 0 < |\epsilon_3| < \Delta t$ ，则 $0 < |\epsilon_1 + \epsilon_3| < 2\Delta t$ ，步长取 $\Delta t = 0.005s$ 时，总误差控制在 $0.01s$ 内。

4.2.3 求解结果

经过模拟得到如下结果：首次实现有效遮蔽时刻为 $8.0425s$ ，最后一次实现有效遮蔽时刻为 $9.4475s$ ，有效遮蔽时长为 $1.4050s$ 。

五、 问题二的求解

5.1 模型构建

针对问题二，本文在问题一所建立的模型基础上，引入无人机 FY1 航向角、飞行速度大小、烟幕弹投放时刻与延迟起爆时间等决策变量，从而实现对有效遮蔽时长的最大化。

5.1.1 决策变量

为了刻画烟幕弹投放策略的决策逻辑，本文定义以下变量作为决策变量：

- a.) θ_1 : 无人机 FY1 的航向角，即与 x 轴正方向的夹角；
- b.) v_1 : 无人机 FY1 的飞行速度大小；
- c.) $t_{1,1}^d$: 无人机 FY1 投放烟幕弹的时刻；
- d.) $\tau_{1,1}$: 无人机 FY1 投放烟幕弹的延迟起爆时间。

5.1.2 导弹轨迹的计算

导弹始终保持以恒定速度 v_m 指向假目标。因此，其飞行规律与问题一相同，运动表达式仍由式(1)(2)给出。

5.1.3 烟幕弹遮蔽范围

与问题一中模型相类似，本问关于烟幕弹遮蔽范围的计算依然分为：计算无人机 FY1 轨迹与投放点坐标；计算弹体落体轨迹与起爆点坐标；计算烟幕云团中心下沉轨迹。

Step.1: 无人机 FY1 轨迹与投放点坐标的计算

本问中，无人机 FY1 飞行方向可变化，但需约束在水平 xoy 平面内。故无人机 FY1 单位航向向量与无人机 FY1 在 t 时刻的位置为及时间为 $t_{1,1}^d$ 时投放点的坐标分别由式(3)(5)(6)给出。

Step.2: 弹体落体轨迹与起爆点坐标的计算

烟幕弹在释放瞬间继承无人机 FY1 飞行时的水平速度，竖直方向仅受重力加速度 g 作用，因此在空间内做水平初速度为 v_1 的平抛运动。设起爆延时为 $\tau_{1,1}$ ，对问题一模型进行推广，则起爆点为

$$\mathbf{p}_{1,1}^b = (x_{1,1}^d + v_1 \cos \theta_1 \tau_{1,1}, y_{1,1}^d + v_1 \sin \theta_1 \tau_{1,1}, z_{1,1}^d - \frac{1}{2}g\tau_{1,1}^2), \quad (20)$$

其中 $(x_{1,1}^d, y_{1,1}^d, z_{1,1}^d)$ 为投放点坐标。该式为问题一式(10)的推广形式。

Step.3: 云团中心下沉轨迹的计算

烟幕弹起爆后，瞬时形成球状云团，并以固定沉降速度 v_s 沿 z 轴负方向匀速运动，其运动规律仍由式(12)给出。

5.1.4 目标函数

为了得到最佳的投放策略，本问的优化目标设定为最大化烟幕对真目标的有效遮蔽时长。首先定义遮蔽指示函数：

$$\chi(t) = \begin{cases} 1, & \text{若目标在时刻 } t \text{ 被有效遮蔽,} \\ 0, & \text{若目标在时刻 } t \text{ 不被有效遮蔽} \end{cases} \quad (21)$$

由此，总有效遮蔽时长可表示为

$$T_{\text{cover}} = \int_{t_{1,1}^b}^{t_{1,1}^b + T_e} \chi(t) dt, \quad (22)$$

其中 $t_{1,1}^b$ 为烟幕起爆时刻， T_e 为计算时间窗口。

在数值计算中，对时间区间进行步长为 Δt 的离散化，令

$$t_n = t_{1,1}^b + n\Delta t, \quad n = 0, 1, \dots, N, \quad N = \frac{T_e}{\Delta t}, \quad (23)$$

则总遮蔽时长可近似为

$$T_{\text{cover}} = \sum_{n=0}^N \chi(t_n) \Delta t. \quad (24)$$

其中本问中的时间步长与问题一保持一致， $\Delta t = 0.005s$ 。

本问的目标函数为最大化离散化近似得到的 T_{cover} ，即通过逐时刻判定目标是否被有效遮蔽并累加，从而获得总遮蔽时长。

5.1.5 约束条件

本优化问题所涉及的约束条件详见表2。这五项约束条件共同确保了无人机航向角始终处于 0° 至 360° 的有效范围内，飞行速度严格限制在题目给定的合理区间（70–140 m/s ）内，同时保证烟幕弹的投放时刻、烟幕弹的起爆延时和烟幕弹的起爆高度均为非负值。

表 2 问题二的约束条件

约束名称	数学表达式
无人机航向角范围约束	$0 \leq \theta_1 < 2\pi$
无人机飞行速度大小范围约束	$v_{\min} \leq v_1 \leq v_{\max}$
烟幕弹投放时刻非负约束	$t_{1,1}^d \geq 0$
烟幕弹延时起爆时间非负约束	$\tau_{1,1} \geq 0$
烟幕弹起爆高度非负约束	$z_{1,1}^b \geq 0$

5.1.6 遮蔽时间最大化模型

综上所述，根据上述公式的建模结果，问题二可形式化为如下最优化问题：

$$\begin{aligned}
 & \max_{\theta_1, v_1, t_{1,1}^d, \tau_{1,1}} T_{\text{cover}}(\theta_1, v_1, t_{1,1}^d, \tau_{1,1}) \\
 & \text{s.t.} \quad 0 \leq \theta_1 < 2\pi, \\
 & \quad v_{\min} \leq v_1 \leq v_{\max}, \\
 & \quad t_{1,1}^d \geq 0, \\
 & \quad \tau_{1,1} \geq 0, \\
 & \quad z_{1,1}^b \geq 0,
 \end{aligned} \tag{25}$$

其中， T_{cover} 为总遮蔽时间，由式(21) ~ (24)离散计算得到。式(25)即为问题二的最优化模型。

5.2 基于遗传算法 GA 的求解

5.2.1 算法描述

针对式(25)所建立的多变量非线性优化模型，本文选择**遗传算法 (Genetic Algorithm, GA)** 作为求解方法^[1]。这一选择主要基于以下考虑：

1. **目标函数非线性特性**：目标函数 T_{cover} 涉及复杂的运动学计算和几何判断，具有强烈的非线性特征。
2. **搜索空间大**：决策变量包含航向角、速度、投放时刻和起爆时间四个维度，且各变量取值范围较广，构成了一个较大的连续搜索空间，需要高效的全局搜索算法。
3. **目标函数多峰特性**：在整个搜索空间中，只有当烟幕弹的投放参数（航向、速度、时机）与导弹运动精确匹配时，才能产生有效遮蔽。这使得目标函数在绝大多数区域值为零（即无效遮蔽），因此显然存在多个局部最优解。
4. **梯度信息缺失**：本优化问题的核心在于对有效遮蔽状态的离散判断（ $\chi(t)$ 函数），此过程本质上是一个逻辑判定而非连续可微的数学表达式，导致目标函数 T_{cover} 无法提供有效的梯度信息。

遗传算法通过模拟自然进化过程，采用选择、交叉、变异等操作，能够在复杂的解空间中有效地进行全局搜索，避免陷入局部最优。在本问题中，遗传算法实现的具体过程分为以下四步：

1. **决策变量编码**：将无人机 FY1 的飞行策略参数编码为决策变量，表示为四维向量 $\mathbf{x} = [\theta_1, v_1, t_{1,1}^d, \tau_{1,1}]$ 。
2. **搜索空间界定**：根据表2中规定的约束条件，限定变量的上下界。

$$\text{下界: } \mathbf{lb} = [0, 70, 0, 0]$$

$$\text{上界: } \mathbf{ub} = [2\pi, 140, 40, 40]$$

3. **初始种群生成**：采用混合初始化策略，首个个体的参数初始值采用问题一的最优飞行参数 $[3.1230, 120, 1.5, 3.6]$ ，其余 199 个个体在搜索空间内均匀随机生成。
4. **目标函数**：以目标函数 $F = -T_{\text{cover}}$ 作为优化目标，通过遗传算法最小化 $-T_{\text{cover}}$ 来等效实现遮蔽时间的最大化。

其中，真目标是否被有效遮蔽的判断方法与问题一中所采用的方法保持一致。遗传算法的参数配置如下：种群规模为 200，最大进化代数为 500,000。本文中问题二至问题五所涉及的算法参数及变量初值设置详见附录 B。

5.2.2 求解结果

经过遗传算法优化，得到最优投放策略变量如表3所示：

表 3 最优投放策略决策变量组合

变量名称	符号	最优值	物理意义
航向角	θ_1	3.1230 rad	无人机飞行方向 (约 178.9°)
飞行速度	v_1	99.2020 m/s	无人机飞行速度大小
投放时刻	$t_{1,1}^d$	0.8094 s	烟幕弹投放时间
起爆延时	$\tau_{1,1}$	3.3250 s	投放至起爆的时间间隔

在该最优参数组合下，获得的最大有效遮蔽时长为：

$$T_{\text{cover}} = 4.5000 \text{ s}$$

本问中求得的最大有效遮蔽时长远大于问题一中固定参数策略得到的有效遮蔽时长。

六、问题三的求解

6.1 模型构建

本问在问题二的建模的基础上，将单无人机-单烟幕弹模型扩展至单无人机-多烟幕弹场景，将决策变量及相应约束条件做相应调整，整体建模思路与前文保持一致。

6.1.1 决策变量

本问中，无人机 FY1 需要投放 3 枚烟幕弹，以尽可能延长对来袭导弹 M1 的遮蔽时间。与问题二相比，主要区别在于投放数量由 1 枚扩展为 3 枚，因此模型的决策变量由单个投放时刻与延时时长扩展为 3 组。

此时，决策变量为：

- a.) θ_1 : 无人机 FY1 的航向角，即与 x 轴正方向的夹角；
- b.) v_1 : 无人机 FY1 的飞行速度大小；
- c.) $\{t_{1,k}^d\}_{k=1}^3$: 无人机 FY1 投放第 k 枚烟幕弹的时刻， $k \in \{1, 2, 3\}$ ；
- d.) $\{\tau_{1,k}\}_{k=1}^3$: 无人机 FY1 投放第 k 枚烟幕弹的延迟起爆时间， $k \in \{1, 2, 3\}$ 。

为便于后续建立最优化目标函数，本文将此若干决策变量写成如下的决策向量形式

$$x_3 = (\theta_1, v_1, \{t_{1,k}^d\}_{k=1}^3, \{\tau_{1,k}\}_{k=1}^3), \quad (26)$$

6.1.2 模型主体

将问题二的模型推广至本问条件，即将无人机投放烟幕弹的数量由单枚扩展为多枚。导弹 M1 与无人机 FY1 的运动逻辑与问题一、二保持一致，以下分别给出第 k 个烟幕弹的投放位置坐标，起爆位置坐标，起爆时刻以及形成球状烟幕云团球心的位置坐标公式。

$$\mathbf{p}_{1,k}^d = \mathbf{A}_1(t_{1,k}^d) \quad (27)$$

$$t_{1,k}^b = t_{1,k}^d + \tau_{1,k} \quad (28)$$

$$\mathbf{p}_{1,k}^b = (x_{1,k}^d + v_1 \cos \theta_1 \tau_{1,k}, y_{1,k}^d + v_1 \sin \theta_1 \tau_{1,k}, z_{1,k}^d - \frac{1}{2}g\tau_{1,k}^2) \quad (29)$$

$$\mathbf{C}_{1,k}(t) = \mathbf{p}_{1,k}^b - (0, 0, v_s(t - t_{1,k}^b)), \quad t \in [t_{1,k}^b, t_{1,k}^b + T_e] \quad (30)$$

其中， k 为投放烟幕弹的序号， $k \in \{1, 2, 3\}$ 。

6.1.3 有效遮蔽判定

在本问中，由于烟幕弹由一枚推广到了多枚，问题一、二中的有效遮蔽判断方法不再有效。如图10所示，在 t 时刻，圆柱形上底面 C_t 被云团 $S_1(t)$ 有效遮蔽，下底面 C_b 被云团 $S_2(t)$ 有效遮蔽，但显然圆柱中间部分没有被有效遮蔽，因此问题一、二中有有效遮蔽判断方法失效。

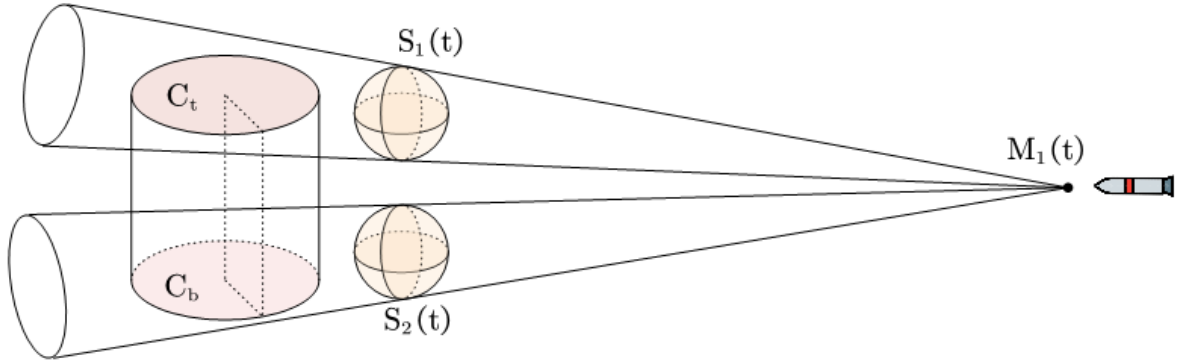


图 10 无人机飞行方向俯视图

针对问题一、二中有有效遮蔽的解析判断法失效问题，本文在问题三、四、五中均采用采样法判断真目标是否被有效遮蔽。具体步骤如下（整个流程如图11所示）：

1. 均匀生成 640 个采样点（计数变量初始设置 $n = 1$ ）。
2. 将第 n 个采样点与导纳位置连线。
3. 分别过三个球心向该连线做垂线，记录三条垂线段的长度分别为 h_1, h_2, h_3 。
4. 判断是否所有垂线段长度均小于球半径 R_s （即 $h_1 < R_s, h_2 < R_s, h_3 < R_s$ ）：

- 如果**不满足**条件，则该时刻**不是有效遮蔽**，流程结束。
- 如果**满足**条件，则继续执行下一步。

5. 判断计数变量 n 的值：

- 如果 $n < 640$ ，则令 $n = n + 1$ 并返回步骤 2。
- 如果 $n = 640$ ，则该时刻判定为**有效遮蔽**，流程结束。

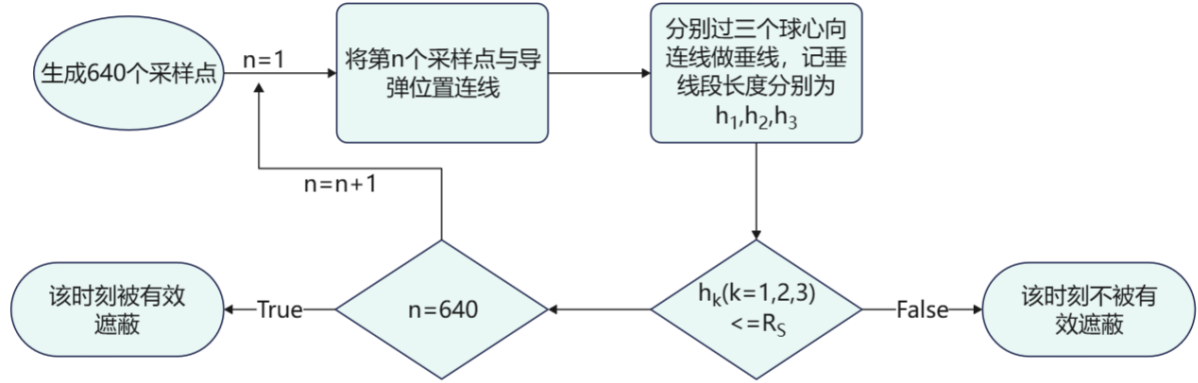


图 11 有效遮蔽判定流程图

6.1.4 目标函数

在问题二中，本文已定义了遮蔽指示函数 $\chi(t)$ 及总遮蔽时长 T_{cover} 的计算方法（见式(21) ~ (24)），并通过数值离散化实现了遮蔽时长的近似计算。

问题三的目标函数在原理上与问题二保持一致，依然为最大化烟幕对真目标的总遮蔽时长。而本问需同时考虑多枚烟幕弹的投放与起爆效果，因此计算区间应从**第一枚烟幕弹起爆时刻**开始，直至**最后一枚烟幕弹的作用时段结束**。在每一离散时刻 t_n ，若真目标被至少一枚烟幕完全遮蔽，则记 $\chi(t_n) = 1$ ，否则 $\chi(t_n) = 0$ 。

因此，本问的目标函数表示为式(31)的形式，但其判定条件由单一烟幕扩展为多烟幕的联合判定。

$$T_{\text{cover}}(x_3). \quad (31)$$

6.1.5 约束条件

本问中模型调整后的约束条件如表4所示。

表 4 问题三的约束条件

约束名称	数学表达式
无人机航向角范围约束	$0 \leq \theta_1 < 2\pi$
无人机速度范围约束	$v_{\min} \leq v_1 \leq v_{\max}$
烟幕弹投放时序约束	$t_{1,k}^d \geq 0, \quad t_{1,k+1}^d - t_{1,k}^d \geq 1$
烟幕弹延时起爆时间非负约束	$\tau_{1,k} \geq 0, \quad k \in \{1, 2, 3\}$
烟幕弹起爆高度非负约束	$z_{1,k}^b \geq 0, \quad k \in \{1, 2, 3\}$

问题三的约束条件即为问题一中对一个烟幕弹的约束条件推广到对三个烟幕弹的约束条件，具体见表4。

6.1.6 遮蔽时间最大化优化模型

综上所述，依据上述决策变量、主体公式、目标函数与约束条件，本问的最优化模型如下

$$\begin{aligned}
 & \max_{x_3} T_{\text{cover}}(x_3) \\
 & \text{s.t.} \quad 0 \leq \theta_1 < 2\pi, \\
 & \quad v_{\min} \leq v_1 \leq v_{\max}, \\
 & \quad t_{1,k}^d \geq 0, \quad t_{1,k+1}^d - t_{1,k}^d \geq 1, \\
 & \quad \tau_{1,k} \geq 0, \quad z_{1,k}^b \geq 0,
 \end{aligned} \tag{32}$$

其中， $k \in \{1, 2, 3\}$ ， $x_3 = (\theta_1, v_1, \{t_{1,k}^d\}_{k=1}^3, \{\tau_{1,k}\}_{k=1}^3)$

6.2 模型求解

6.2.1 基于遗传算法求解最长有效遮蔽时长

本问同样使用遗传算法（GA）求解最优投放策略（决策变量初始值以及遗传算法参数设置见附录 B），但对真目标是否被有效遮蔽的判定采用采样法判断，具体判断流程如图11所示。

经过遗传算法优化，得到最优投放策略中航向角 $\theta_1 = 179.5595^\circ$ ，飞行速度大小 $v_1 = 121.2417m/s$ ，三个烟幕弹的投放时刻与起爆延时见表5。

表 5 烟幕弹投放决策变量

烟幕弹编号	投放时刻 (s)	起爆延时 (s)
烟幕弹 1	0.0993	3.4747
烟幕弹 2	3.1929	4.8082
烟幕弹 3	5.1680	5.4649

得到最长有效遮蔽时长为 6.8800s。

6.2.2 最优投放策略结果分析

通过优化求解发现，无人机的最优航向角 θ_1 接近 180° ，表明其飞行方向与导弹飞行方向在水平面内的投影高度一致，且略偏向真目标所在方位。这一结果表明，在单无人机-单导弹场景下，且二者初始位置较为接近时，无人机的最优飞行策略为尽可能与导弹保持同向飞行，并在水平方向上微调以对准真目标，从而最大化烟幕弹的遮蔽时长。

6.2.3 每个烟幕弹的有效遮蔽时长

得到最优投放策略后，为了计算每个烟幕弹单独为真目标提供有效遮蔽的时长，将得到的无人机航向角和飞行速度大小及每个烟幕弹的投放参数带入问题一的模型中，得到结果如表8所示。本问题中所有结果整合见附件 result1.xlsx。

表 6 单个烟幕弹提供的有效遮蔽时长

烟幕弹编号	有效遮蔽时长 (s)
烟幕弹 1	3.4150
烟幕弹 2	2.6400
烟幕弹 3	1.0600

6.2.4 结果可靠性分析

在计算每个烟幕弹单独为真目标提供有效遮蔽的时长，问题一中的有效遮蔽判断法与本问题中的采样法同样适用。本文用两种方法分别对每个烟幕弹单独的有效遮蔽时长进行计算，两种判断方法计算得出的有效遮蔽时长一致，从而避免了采样点数量不足导致的结果不准确。具体代码见附录。

七、问题四的求解

7.1 模型构建

本问在问题二的建模的基础上，将单无人机-单烟幕弹模型扩展至多无人机-单烟幕弹场景，将决策变量及相应约束条件做相应调整，整体建模思路与前文保持一致。

7.1.1 决策变量

对模型进行推广后，本模型决策向量设置如下：

$$x_4 = (\{\theta_j\}_{j=1}^3, \{v_j\}_{j=1}^3, \{t_{j,1}^d\}_{j=1}^3, \{\tau_{j,1}\}_{j=1}^3), \quad (33)$$

7.1.2 模型主体

将问题二中的单无人机-单烟幕弹模型扩展至多无人机-单烟幕弹场景后，各物体的运动学方程如下。

$$\vec{\mathbf{m}}_1 = \frac{\mathbf{0} - \mathbf{M}_1}{\|\mathbf{0} - \mathbf{M}_1\|} \quad (34)$$

$$\mathbf{M}_1(t) = \mathbf{M}_1 + v_m t \vec{\mathbf{m}}_1 \quad (35)$$

$$\vec{\mathbf{a}}_j(\theta) = (\cos \theta_j, \sin \theta_j, 0) \quad (36)$$

$$\mathbf{A}_j(t) = \mathbf{A}_j + v_j t \vec{\mathbf{a}}_j(\theta) \quad (37)$$

$$\mathbf{p}_{j,1}^d = \mathbf{A}_j(t_{j,1}^d) \quad (38)$$

$$t_{j,1}^b = t_{j,1}^d + \tau_{j,1} \quad (39)$$

$$\mathbf{p}_{j,1}^b = (x_{j,1}^d + v_j \cos \theta_j \tau_{j,1}, y_{j,1}^d + v_j \sin \theta_j \tau_{j,1}, z_{j,1}^d - \frac{1}{2} g \tau_{j,1}^2) \quad (40)$$

$$\mathbf{C}_{j,1}(t) = \mathbf{p}_{j,1}^b - (0, 0, v_s(t - t_{j,1}^b)), \quad t \in [t_{j,1}^b, t_{j,1}^b + T_e] \quad (41)$$

其中， j 为无人机的序号， $j \in \{1, 2, 3\}$ 。

7.1.3 目标函数

本问的目标函数在原理上与问题三完全一致，依然是最大化总遮蔽时长，区别仅在于本问所考虑的烟幕弹集合不同，即烟幕弹编号 j, k 的取值范围发生改变。因而，本问的优化仍直接采用问题三的目标函数形式，只需调整相应的烟幕弹范围，即式(42)。

$$T_{\text{cover}}(x_4). \quad (42)$$

7.1.4 约束条件

本问中模型调整后的约束条件如表7所示。

表 7 问题四的约束条件

约束名称	数学表达式
无人机航向角范围约束	$0 \leq \theta_j < 2\pi$
无人机速度范围约束	$v_{\min} \leq v_j \leq v_{\max}$
烟幕弹投放时刻非负约束	$t_{j,1}^d \geq 0$
烟幕弹延时起爆时间非负约束	$\tau_{j,1} \geq 0$
烟幕弹起爆高度非负约束	$z_{j,1}^b \geq 0$

* j 为无人机的序号, $j \in \{1, 2, 3\}$.

7.1.5 最优化模型

综上所述, 本问的最优化模型如下

$$\begin{aligned}
 \max_{x_4} \quad & T_{\text{cover}}(x_4) \\
 \text{s.t.} \quad & 0 \leq \theta_j < 2\pi, \\
 & v_{\min} \leq v_j \leq v_{\max}, \\
 & t_{j,1}^d \geq 0, \quad \tau_{j,1} \geq 0, \quad z_{j,1}^b \geq 0,
 \end{aligned} \tag{43}$$

其中, $j \in \{1, 2, 3\}$, $x_4 = (\{\theta_j\}_{j=1}^3, \{v_j\}_{j=1}^3, \{t_{j,1}^d\}_{j=1}^3, \{\tau_{j,1}\}_{j=1}^3)$ 。

7.2 模型求解

与问题三中的模型求解方法一致, 先通过**遗传算法 (GA)** 求出最优投放策略, 再带入最优策略中的参数, 计算每个烟幕弹单独为真目标提供有效遮蔽的时长, 最后分别使用两种有效遮蔽判断方法交叉验证结果可靠性。

7.2.1 遗传算法初值选取策略

由于本问题中的搜索空间规模较大, 且空间内存在大量目标函数值为零的点, 若采用随机初始化策略, 可能导致遗传算法在初始阶段难以获得有效的初始解, 从而使其在较长时间内无法跳出低适应度区域, 进而影响算法收敛到最优解的效率。

为解决上述问题, 首先针对每架无人机仅投放一枚烟幕弹的情形, 将其代入问题二的模型进行求解, 再将所得结果作为本问题中求解策略参数的初始值。具体初始化数值见附录 B。

7.2.2 求解结果

经过遗传算法优化，得到本问最优投放策略。其无人机航向角 θ_j ，飞行速度 v_j ，各烟幕弹投放时刻与起爆延时见表8与表9。其中各烟幕弹单独有效遮蔽时长经交叉验证，得到两种判断方法所得遮蔽时长完全一致。

表 8 无人机决策变量

无人机编号	无人机航向角 (°)	无人机飞行速度 (m/s)
FY1	178.8847	101.1553
FY2	233.9977	123.8281
FY3	115.9023	123.7188

表 9 烟幕弹投放决策变量

烟幕弹编号	投放时刻 (s)	起爆延时 (s)	有效遮蔽时长 (s)
烟幕弹 1	0.8000	3.3055	4.4750
烟幕弹 2	6.5000	7.3000	3.4500
烟幕弹 3	21.0000	7.0175	2.7350

得到最长有效遮蔽时长为 10.6500s。问题四的所有求解结果整合见附件 result2.xlsx。

7.2.3 最优投放策略结果分析

通过优化结果分析发现，无人机 FY1、FY2、FY3 与导弹的初始距离依次增大，其对应的烟幕弹投放时刻也依次延迟。这一现象表明，在单无人机-单烟幕弹-单导弹的场景中，烟幕弹的最优投放时刻与无人机-导弹初始距离之间存在正相关关系。从运动学角度分析，初始距离较大的无人机需要更长的航行时间才能到达合适的投放点，因此其最优投放时刻相应延后。

八、 问题五的求解

8.1 模型构建

本问综合考虑在多架无人机分别投放多枚烟幕弹对于多枚导弹，真目标能够被有效遮蔽的最优投放策略。在问题二、三、四的基础上建立多无人机-多烟幕弹模型。

8.1.1 决策变量

面对三枚来袭导弹，使用五架无人机分别最多投放三枚烟幕弹的情况，模型的决策向量定义如下：

$$x_5 = (\{\theta_j\}_{j=1}^5, \{v_j\}_{j=1}^5, \{t_{j,k}^d\}_{j=1,k=1}^{5,3}, \{\tau_{j,k}\}_{j=1,k=1}^{5,3}) \quad (44)$$

8.1.2 模型主体

与前文模型原理一致，本模型在推广后如下：

$$\vec{\mathbf{m}}_i = \frac{\mathbf{0} - \mathbf{M}_i}{\|\mathbf{0} - \mathbf{M}_i\|} \quad (45)$$

$$\mathbf{M}_i(t) = \mathbf{M}_i + v_m t \vec{\mathbf{m}}_i \quad (46)$$

$$\vec{\mathbf{a}}_j(\theta) = (\cos \theta_j, \sin \theta_j, 0) \quad (47)$$

$$\mathbf{A}_j(t) = \mathbf{A}_j + v_j t \vec{\mathbf{a}}_j(\theta) \quad (48)$$

$$\mathbf{p}_{j,k}^d = \mathbf{A}_j(t_{j,k}^d) \quad (49)$$

$$t_{j,k}^b = t_{j,k}^d + \tau_{j,k} \quad (50)$$

$$\mathbf{p}_{j,k}^b = (x_{j,k}^d + v_j \cos \theta_j \tau_{j,k}, y_{j,k}^d + v_j \sin \theta_j \tau_{j,k}, z_{j,k}^d - \frac{1}{2} g \tau_{j,k}^2) \quad (51)$$

$$\mathbf{C}_{j,k}(t) = \mathbf{p}_{j,k}^b - (0, 0, v_s(t - t_{j,k}^b)), \quad t \in [t_{j,k}^b, t_{j,k}^b + T_e] \quad (52)$$

其中， i 为来袭导弹的序号， $i \in \{1, 2, 3\}$ ； j 为无人机的序号， $j \in \{1, 2, 3, 4, 5\}$ ； k 为某无人机投下的烟幕弹的序号， $k \in \{1, 2, 3\}$ 。

8.1.3 目标函数

本模型的目标函数亦为最大化总遮蔽时长。面对导弹、无人机与烟幕弹数量均大于 1 的情况下，为保证总体遮蔽时间最长，本模型将三枚导弹的遮蔽时长之和作为目标函数。对于每枚导弹采用与前文一致的方法，对时间进行离散化，随后进行累加。

$$t_n^i = t_{j,k}^b + n \Delta t, \quad n = 0, \dots, N, \quad N = \frac{T_e}{\Delta t} \quad (53)$$

$$\chi_i(t) = \begin{cases} 1, & \text{对于第 } i \text{ 枚导弹真目标被有效遮蔽,} \\ 0, & \text{否则.} \end{cases} \quad (54)$$

$$T_{\text{cover}}(x_5) = \sum_{i=1}^3 \sum_{n=0}^N \chi_i(t_n^i) \Delta t. \quad (55)$$

8.1.4 约束条件

本问中模型调整后的约束条件如表10所示。

表 10 问题五的约束条件

约束名称	数学表达式
无人机航向角范围约束	$0 \leq \theta_j < 2\pi$
无人机速度范围约束	$v_{\min} \leq v_j \leq v_{\max}$
烟幕弹投放时序约束	$t_{j,k}^d \geq 0, \quad t_{j,k+1}^d - t_{j,k}^d \geq 1$
烟幕弹延时起爆时间非负约束	$\tau_{j,k} \geq 0$
烟幕弹起爆高度非负约束	$z_{j,k}^b \geq 0$

* 其中, $j \in \{1, 2, 3, 4, 5\}$, $k \in \{1, 2, 3\}$

8.1.5 最优化模型

综上所述, 本问的最优化模型即为

$$\begin{aligned}
 \max_{x_5} \quad & T_{\text{cover}}(x_5) \\
 \text{s.t.} \quad & 0 \leq \theta_j < 2\pi, \\
 & v_{\min} \leq v_j \leq v_{\max}, \\
 & t_{j,k}^d \geq 0, \quad t_{j,k+1}^d - t_{j,k}^d \geq 1, \\
 & \tau_{j,k} \geq 0, \quad z_{j,k}^b \geq 0,
 \end{aligned} \tag{56}$$

其中, $j \in \{1, 2, 3, 4, 5\}$, $k \in \{1, 2, 3\}$, $x_5 = (\{\theta_j\}_{j=1}^5, \{v_j\}_{j=1}^5, \{t_{j,k}^d\}_{j=1, k=1}^{5,3}, \{\tau_{j,k}\}_{j=1, k=1}^{5,3})$

8.2 模型求解

与前问中的模型求解方法一致, 先通过遗传算法 (GA) 求出最优投放策略, 随后带入最优策略中的参数, 计算每个烟幕弹单独为真目标提供有效遮蔽的时长并加和, 最后分别使用两种有效遮蔽判断方法交叉验证结果可靠性。

8.2.1 遗传算法初值选取策略

由于本问中决策变量数量显著增加 (共 40 个), 搜索空间极为庞大, 因此本问面临与问题四同样的遗传算法的初值选取问题。

考虑到本问中存在 3 枚导弹, 且每架无人机需要投放三枚烟幕弹, 本文对问题三中单架无人机投放三枚烟幕弹的求解过程稍作修改, 将优化目标函数对应改为与本问相同

的三枚导弹情形。之后分别将五架无人机的坐标代入修改后的问题三模型中，将求解结果作为本问求解的初值。具体初始化数值见附录 B。

8.2.2 求解结果

通过遗传算法优化后即得到本问最优投放策略。考虑到参数过多，正文中仅选取无人机 FY1 的投放策略参数进行展示，其余结果见附件 result3.xlsx。

无人机 FY1 的投放策略中航向角 $\theta_1 = 179.4571^\circ$ ，飞行速度 $v_1 = 121.5454m/s$ ，无人机 FY1 投放的三个烟幕弹的投放时刻、起爆延时与有效遮蔽时长见表11。

表 11 问题五烟幕弹投放决策变量

烟幕弹编号	投放时刻 (s)	起爆延时 (s)	有效遮蔽时长 (s)
烟幕弹 1	0.1048	3.4897	3.3500
烟幕弹 2	3.1528	4.5851	2.4650
烟幕弹 3	5.1522	5.5055	1.0550

最终得到最长有效遮蔽总时间为 31.0800s。

8.2.3 最优化策略结果分析

单个烟幕弹针对单个导弹的有效遮蔽时长热力图如图12所示，其中烟幕弹 1 号至 3 号由无人机 FY1 投放，烟幕弹 4 号至 6 号由无人机 FY2 投放，依次类推。

	1号烟幕弹	2号烟幕弹	3号烟幕弹	4号烟幕弹	5号烟幕弹	6号烟幕弹	7号烟幕弹	8号烟幕弹	9号烟幕弹	10号烟幕弹	11号烟幕弹	12号烟幕弹	13号烟幕弹	14号烟幕弹	15号烟幕弹
1号导弹	3.35	2.47	1.06	0	0	3.1	0	0	0	0	0	0	0	3.6	0
2号导弹	0	0	0	0	3.51	0	0	2.57	0	3.35	0	0	0	0	1.97
3号导弹	0	0	0	0	0	0	2.5	0	0	0	2.5	0	2.79	0	0

图 12 烟幕弹针对导弹有效遮蔽时长热力图

在多个导弹来袭的情况下，本文得到了两种无人机最优投放策略。第一种策略以无人机 FY1 为代表，适用于无人机初始位置邻近某枚导弹飞行轨迹的情形。该类无人机所投放的烟幕弹主要用于遮蔽距离最近的导弹，实现对该目标的定向干扰。

若无人机不满足第一种策略的条件，则采用第二种策略，即应尽可能使得其飞行路径的水平投影与多枚导弹的飞行轨迹的水平投影存在在多个位置相交（以无人机 FY5 为例）。该类无人机通过规划航迹使其尽可能覆盖多个导弹的预计交汇区域，从而在多个时空点上形成有效遮蔽，进而最大化有效遮蔽时长。

九、模型评价

9.1 模型的优点

1. **对有效遮蔽的判定意义明确。**本文基于几何光学原理，提出了以四次多项式全局非负性为判据的有效遮蔽判定的几何解析法，具有明确的数学基础和物理意义，确保了判断过程的严谨性与可解释性。
2. **双重验证机制。**在复杂场景（如多烟幕弹联合遮蔽）中，针对单个烟幕弹提供的有效遮蔽时长的结果，本文进行了几何解析法与采样法的交叉检验，两种方法结果一致，说明模型计算结果准确性较高。

9.2 模型的不足

1. **算法对初始值的依赖程度较大。**遗传算法的收敛速度与最优解的质量对初始值的选择依赖性较高，若初始值离最优区域较远，易陷入局部最优，影响优化效果与算法收敛速度。

9.3 模型的推广

本文提出的针对有效遮蔽的几何解析法与最大有效遮蔽时长的优化框架可推广至红外干扰、箔条云干扰等其他电子对抗手段的建模与优化中，具有广泛的军事应用前景。

参考文献

- [1] 葛继科, 邱玉辉, 吴春明, 等. 遗传算法研究综述 [J]. 计算机应用研究, 2008, (10): 2911-2916.
- [2] 陈浩, 高欣宝, 李天鹏, 姚强, 魏咏梅. 国外烟幕干扰弹发展及关键技术研究 [J]. 飞航导弹, 2017, (12): 71-74. DOI: 10.16338/j.issn.1009-1319.2017.12.16.
- [3] 姜启源, 谢金星, 叶俊. 数学模型（第六版）[M]. 高等教育出版社, 北京, 2024.

注：本参赛队未使用任何 AI 工具。

附录 A 支撑材料文件列表

表 12 支撑材料文件列表

文件名	文件说明
result1.xlsx	问题 3 计算结果数据
result2.xlsx	问题 4 计算结果数据
result3.xlsx	问题 5 计算结果数据
q1.m	问题 1 仿真与优化代码
q2.m	问题 2 仿真与优化代码
q3.m	问题 3 仿真与优化代码
q3_s.m	问题 3 结果分析代码
q4.m	问题 4 仿真与优化代码
q4_s.m	问题 4 结果分析代码
q5.m	问题 5 仿真与优化代码
q5_s.m	问题 5 结果分析代码

附录 B 各程序的初值选取与参数设置

表 13 问题 2-5 变量及 GA 参数汇总

变量	问题 2	问题 3	问题 4	问题 5
θ_1	π	π	π	3.1339
θ_2	—	—	4.0843	3.8574
θ_3	—	—	2.0216	1.7040
θ_4	—	—	—	3.8913
θ_5	—	—	—	2.1148
v_1	120	120	100	121.0000
v_2	—	—	125	120.8979
v_3	—	—	125	120.5277
v_4	—	—	—	114.6142
v_5	—	—	—	120.5786
t_{11}^d	1.5	0	0.8	0.1
t_{12}^d	—	2	—	3.1
t_{13}^d	—	4	—	5.1

接下页

表 13 – 续表

变量	问题 2	问题 3	问题 4	问题 5
t_{21}^d	—	—	6.5	2.8133
t_{22}^d	—	—	—	6.4717
t_{23}^d	—	—	—	8.8673
t_{31}^d	—	—	21	19.9799
t_{32}^d	—	—	—	23.4986
t_{33}^d	—	—	—	28.8131
t_{41}^d	—	—	—	8.6996
t_{42}^d	—	—	—	15.6559
t_{43}^d	—	—	—	16.7156
t_{51}^d	—	—	—	13.7658
t_{52}^d	—	—	—	14.7926
t_{53}^d	—	—	—	20.9975
t_{11}^b	3.6	2	3.3	3.5
t_{12}^b	—	2	—	4.5
t_{13}^b	—	2	—	5.5
t_{21}^b	—	—	7.3	6.00
t_{22}^b	—	—	—	6.25
t_{23}^b	—	—	—	8.28
t_{31}^b	—	—	7	4.21
t_{32}^b	—	—	—	3.78
t_{33}^b	—	—	—	2.64
t_{41}^b	—	—	—	12.5674
t_{42}^b	—	—	—	13.4961
t_{43}^b	—	—	—	6.67
t_{51}^b	—	—	—	1.8331
t_{52}^b	—	—	—	4.8365
t_{53}^b	—	—	—	1.9457
GA 参数				
决策变量个数	4	8	12	40
种群大小	200	200	200	100
最大迭代次数	500000	5000	2000	2000

注：为方便编码需要，本表中的角度单位均采用弧度制。

附录 C 支撑材料中所有 Matlab 代码

注：运行以下代码时，由于决策变量较多，并且优化函数仅在少数紧密区域中取非零值。为了避免出现结果不良、运行时间过长等问题，请按照附录 B 设置参数，并且可以先取精度不高的参数，如取 $dt=0.5$, $hSamp=2$, $aSamp=4$ ；，以降低仿真的复杂度，取得近似最优解，再以此作为出发点进行较高精度的二次搜索。

3.1 问题 1 仿真代码

```
% q1.m
FY1_0 = [17800, 0, 1800];
M1_0 = [20000, 0, 2000];
v_M1_speed = 300;
v_FY1_speed = 120;
t_release = 1.5;
t_fuse = 3.6;
t_explode = t_release + t_fuse;
r_smoke = 10;
v_smoke_center = [0,0,-3];
C0 = [0,200,0];
H = 10; R = 7; C1 = C0 + [0,0,H];
g = 9.81;
%% 计算无人机直线
dir_FY1 = ( [0,0,1800] - FY1_0 );
dir_FY1 = dir_FY1 / norm(dir_FY1);
v_FY1 = v_FY1_speed * dir_FY1;

%% 计算释放时位置与速度
pos_release = FY1_0 + v_FY1 * t_release;
v_smoke_at_release = v_FY1;
t_fall = t_fuse;
smoke_center_at_explode = pos_release + v_smoke_at_release * t_fall + 0.5 * [0,0,-g] *
    t_fall^2;
v_smoke_at_explode = v_smoke_at_release + [0,0,-g] * t_fall;
dir_M1_to_0 = ( [0,0,0] - M1_0 );
dir_M1_to_0 = dir_M1_to_0 / norm(dir_M1_to_0);
v_M1 = v_M1_speed * dir_M1_to_0;
M1_pos_at_explode = M1_0 + v_M1 * t_explode;

%% 差分模拟
dt = 0.005;
Tspan = 20; % 模拟引爆之后的时间长度（秒）
N = floor(Tspan / dt);
times = (0:N) * dt; % 这是模拟从 0 到 Tspan（相对引爆时刻）
```

```

M1_traj = zeros(N+1,3);
smoke_traj = zeros(N+1,3);
cylinder_inside_cone = false(N+1,1);

M1_traj(1,:) = M1_pos_at_explode;
smoke_traj(1,:) = smoke_center_at_explode;

%% 差分
for n = 2:N+1
    M1_traj(n,:) = M1_traj(n-1,:) + v_M1 * dt;
    smoke_traj(n,:) = smoke_traj(n-1,:) + v_smoke_center * dt;
    abs_t = times(n) + t_explode;
    d = smoke_traj(n,:) - M1_traj(n,:);
    L = norm(d);

    if L > 0
        if L > r_smoke
            u = d / L; % 圆锥轴方向
            alpha = asin(min(1, r_smoke / L));
            cylinder_inside_cone(n) = isCylinderInCone(M1_traj(n,:), C0, C1, R, u, alpha);
            if cylinder_inside_cone(n)
                abs_t = times(n) + t_explode;
            end
        else
            cylinder_inside_cone(n) = true;
        end
    else
        cylinder_inside_cone(n) = true;
    end
end

%% 解析进入/离开时间
abs_times = times + t_explode;
inside = cylinder_inside_cone;
enter_times = [];
exit_times = [];
state_prev = false;

for n = 1:length(inside)
    state = inside(n);
    if ~state_prev && state
        if n==1
            t_enter = abs_times(n);
        else
            t_enter = abs_times(n-1) + dt/2;
        end
        enter_times = [enter_times, t_enter];
    end
end

```

```

elseif state_prev && ~state
    if n==1
        t_exit = abs_times(n);
    else
        t_exit = abs_times(n-1) + dt/2;
    end
    exit_times = [exit_times, t_exit];
end
state_prev = state;
end

%% output
if isempty(enter_times)
    fprintf('在模拟的引爆后 %.4f s 内（到绝对时间 %.4f s）圆柱未被遮挡\n', Tspan, t_explode +
        Tspan);
else
    for k = 1:length(enter_times)
        fprintf('圆柱第 %d 次被遮挡（绝对秒）：%.4f s\n', k, enter_times(k));
        if k <= length(exit_times)
            fprintf('圆柱第 %d 次结束遮挡（绝对秒）：%.4f s\n', k, exit_times(k));
        else
            fprintf('圆柱到模拟结束（绝对时间 %.4f s）仍被遮挡\n', t_explode + Tspan);
        end
    end
end

function isInside = isCylinderInCone(observer_pos, cyl_bottom, cyl_top, cyl_radius, u, alpha)
% 判断：当且仅当圆柱的上下两个水平圆盘（底面/顶面）
% 都完全被以 observer_pos 为顶点、轴方向为 u、半顶角为 alpha 的无限圆锥包含时返回 true。
% observer_pos - 圆锥顶点 (1x3)
% cyl_bottom - 圆柱底面圆心 (1x3)
% cyl_top - 圆柱顶面圆心 (1x3)
% cyl_radius - 圆柱半径（标量）
% u - 圆锥轴单位向量 (1x3)
% alpha - 圆锥半顶角（弧度）
insideB = isDiskInCone_exact(observer_pos, cyl_bottom, cyl_radius, u, alpha);
insideT = isDiskInCone_exact(observer_pos, cyl_top, cyl_radius, u, alpha);
isInside = insideB && insideT;
end

function inside = isDiskInCone_exact(V, C, r, u, alpha)
tol = 1e-12;
negTol = -1e-9;

w = C(:) - V(:);

```

```

s0 = dot(w, u(:));
Ax = u(1); Ay = u(2);
A = Ax; B = Ay;
C1 = w(1); C2 = w(2);
W2 = dot(w,w);
k = (cos(alpha))^2;

amp = r * sqrt(A^2 + B^2);
if s0 - amp < -tol
    inside = false;
    return;
end

% 构造 h(theta) = s(theta)^2 - ||P-V||^2 * cos^2(alpha)
% 化简后得到形式: h = C0 + Cc*cos + Cs*sin + Ccos2*cos2 + Csin2*sin2
C0 = s0^2 - k*(W2 + r^2) + r^2*(A^2 + B^2)/2;
Cc = 2*s0*r*A - 2*k*r*C1;
Cs = 2*s0*r*B - 2*k*r*C2;
Ccos2 = r^2*(A^2 - B^2)/2;
Csin2 = r^2*(A*B);

% 将 h(theta) * (1+t^2)^2 表示为四次多项式 P(t) = c4 t^4 + c3 t^3 + c2 t^2 + c1 t + c0
c0 = C0 + Cc + Ccos2;
c1 = 2*Cs + 4*Csin2;
c2 = 2*C0 - 6*Ccos2;
c3 = 2*Cs - 4*Csin2;
c4 = C0 - Cc + Ccos2;

% 多项式系数向量
p = [c4, c3, c2, c1, c0];

% 若最高阶系数为负, 则多项式在无穷处趋向 -inf, 必有负值 -> 不满足
if c4 < -tol
    inside = false;
    return;
end

% 求导数并找出导数的实根 (候选极值点)
dp = [4*c4, 3*c3, 2*c2, c1];
rts = roots(dp);
cand = [];
for i = 1:numel(rts)
    if abs(imag(rts(i))) < 1e-8
        cand(end+1) = real(rts(i));
    end
end

% 加入 t=0 (对应 theta=0) 的检查点

```

```

cand(end+1) = 0;

% 如果最高阶系数接近零, 说明 P 可能降阶, 为保险起见在很大 t 处再检查一次
if abs(c4) < tol
    T = 1e6;
    cand(end+1) = T;
    cand(end+1) = -T;
end

Pvals = polyval(p,cand);
Pmin = min(Pvals);

% 因为  $h(\theta) = P(t) / (1+t^2)^2$ , 分母始终>0, 所以  $P(t)$  在全实轴非负等价于  $h(\theta)$  对所有  $\theta$  非负
if Pmin < negTol
    inside = false;
else
    inside = true;
end
end
end

```

3.2 问题2 仿真与优化代码

```

%q2.m
% 决策变量: [theta, v_FY1_speed, t_release, t_fuse]
% 目标: 最大化遮蔽时长
clc; clear; close all;
%% GA
nvars = 4;
lb = [0, 70, 0, 0]; % 下界
ub = [2*pi, 140, 40, 40]; % 上界
theta0 = pi; % 参数就是问题一中FY1飞行的角度, 这里硬编码进了代码中
v0 = 120;
t_release0 = 1.5;
t_fuse0 = 3.6;

populationSize = 200;
InitialPopulationMatrix = zeros(populationSize, nvars);
InitialPopulationMatrix(1,:) = [theta0, v0, t_release0, t_fuse0];
for i = 2:populationSize
    InitialPopulationMatrix(i,:) = lb + rand(1,4).*(ub-lb);
end

options = optimoptions('ga','Display','iter','MaxGenerations',500000,'PopulationSize',...
    populationSize,'InitialPopulationMatrix',InitialPopulationMatrix);

```

```

[x_opt,fval] = ga(@(x) -w(x), nvars, [],[],[],[], lb, ub, [], options);

fprintf('\n\n最优解: theta=%.4f rad, v_FY1=%.4f m/s, t_release=%.4f s, t_fuse=%.4f s\n',...
    x_opt(1), x_opt(2), x_opt(3), x_opt(4));
fprintf('最大遮蔽时长 = %.4f s\n', -fval);
%% 目标函数
function Tcover = w(x)
    theta      = x(1);
    v_FY1_speed = x(2);
    t_release   = x(3);
    t_fuse      = x(4);

    FY1_0 = [17800, 0, 1800];
    M1_0 = [20000, 0, 2000];
    v_M1_speed = 300;

    r_smoke = 10;
    v_smoke_center = [0,0,-3];
    C0 = [0,200,0];
    H = 10; R = 7; C1 = C0 + [0,0,H];
    g = 9.81;

    dir_FY1 = [cos(theta), sin(theta), 0];
    v_FY1 = v_FY1_speed * dir_FY1;

    pos_release = FY1_0 + v_FY1 * t_release;
    v_smoke_at_release = v_FY1;

    t_explode = t_release + t_fuse;
    smoke_center_at_explode = pos_release + v_smoke_at_release*t_fuse + 0.5*[0,0,-g]*t_fuse^2;

    dir_M1 = ([0,0,0] - M1_0);
    dir_M1 = dir_M1 / norm(dir_M1);
    v_M1 = v_M1_speed * dir_M1;
    M1_pos_at_explode = M1_0 + v_M1*t_explode;

    dt = 0.005; Tspan = 20;
    N = floor(Tspan/dt);
    M1_pos = M1_pos_at_explode;
    smoke_pos = smoke_center_at_explode;

    covered = false(N,1);

    for n=1:N
        M1_pos = M1_pos + v_M1*dt;
        smoke_pos = smoke_pos + v_smoke_center*dt;
    end
end

```

```

    d = smoke_pos - M1_pos;
    L = norm(d);
    if L > r_smoke
        u = d/L;
        alpha = asin(min(1,r_smoke/L));
        covered(n) = isCylinderInCone(M1_pos,C0,C1,R,u,alpha);
    else
        covered(n) = true;
    end
end

Tcover = sum(covered)*dt;
end

function isInside = isCylinderInCone(observer_pos, cyl_bottom, cyl_top, cyl_radius, u, alpha)
% 判断：当且仅当圆柱的上下两个水平圆盘（底面/顶面）
% 都完全被以 observer_pos 为顶点、轴方向为 u、半顶角为 alpha 的无限圆锥包含时返回 true。
% observer_pos - 圆锥顶点 (1x3)
% cyl_bottom - 圆柱底面圆心 (1x3)
% cyl_top - 圆柱顶面圆心 (1x3)
% cyl_radius - 圆柱半径（标量）
% u - 圆锥轴单位向量 (1x3)
% alpha - 圆锥半顶角（弧度）
insideB = isDiskInCone_exact(observer_pos, cyl_bottom, cyl_radius, u, alpha);
insideT = isDiskInCone_exact(observer_pos, cyl_top, cyl_radius, u, alpha);
isInside = insideB && insideT;
end

function inside = isDiskInCone_exact(V, C, r, u, alpha)
tol = 1e-12;
negTol = -1e-9;

w = C(:) - V(:);
s0 = dot(w, u(:));
Ax = u(1); Ay = u(2);
A = Ax; B = Ay;
C1 = w(1); C2 = w(2);
W2 = dot(w,w);
k = (cos(alpha))^2;

amp = r * sqrt(A^2 + B^2);
if s0 - amp < -tol
    inside = false;
    return;
end

```

```

% 构造  $h(\theta) = s(\theta)^2 - ||P-V||^2 * \cos^2(\alpha)$ 
% 化简后得到形式:  $h = C0 + Cc*\cos + Cs*\sin + Ccos2*\cos2 + Csin2*\sin2$ 
C0 = s0^2 - k*(W2 + r^2) + r^2*(A^2 + B^2)/2;
Cc = 2*s0*r*A - 2*k*r*C1;
Cs = 2*s0*r*B - 2*k*r*C2;
Ccos2 = r^2*(A^2 - B^2)/2;
Csin2 = r^2*(A*B);

% 将  $h(\theta) * (1+t^2)^2$  表示为四次多项式  $P(t) = c4 t^4 + c3 t^3 + c2 t^2 + c1 t + c0$ 
c0 = C0 + Cc + Ccos2;
c1 = 2*Cs + 4*Csin2;
c2 = 2*C0 - 6*Ccos2;
c3 = 2*Cs - 4*Csin2;
c4 = C0 - Cc + Ccos2;

% 多项式系数向量
p = [c4, c3, c2, c1, c0];

% 若最高阶系数为负, 则多项式在无穷处趋向  $-\infty$ , 必有负值 -> 不满足
if c4 < -tol
    inside = false;
    return;
end

% 求导数并找出导数的实根 (候选极值点)
dp = [4*c4, 3*c3, 2*c2, c1];
rts = roots(dp);
cand = [];
for i = 1:numel(rts)
    if abs(imag(rts(i))) < 1e-8
        cand(end+1) = real(rts(i));
    end
end

% 加入  $t=0$  (对应  $\theta=0$ ) 的检查点
cand(end+1) = 0;

% 如果最高阶系数接近零, 说明 P 可能降阶, 为保险起见在很大 t 处再检查一次
if abs(c4) < tol
    T = 1e6;
    cand(end+1) = T;
    cand(end+1) = -T;
end

Pvals = polyval(p, cand);
Pmin = min(Pvals);

```

```

% 因为  $h(\theta) = P(t) / (1+t^2)^2$ , 分母始终>0, 所以  $P(t)$  在全实轴非负等价于  $h(\theta)$  对所有
     $\theta$  非负
if Pmin < negTol
    inside = false;
else
    inside = true;
end
end

Optimization terminated: average change in the fitness value less than
    options.FunctionTolerance.
\n最优解:  $\theta=3.1230$  rad,  $v_{FY1}=99.2020$  m/s,  $t_{release}=0.8094$  s,  $t_{fuse}=3.3250$ 
    s\n最大遮蔽时长 = 4.5000 s\n>>

```

3.3 问题3 仿真与优化代码

```

% q3.m
% 决策变量: [ $\theta$ ,  $v_{FY1\_speed}$ ,  $t_{release0}$ ,  $t_{fuse0}$ ,  $t_{release1}$ ,  $t_{fuse1}$ ,  $t_{release2}$ ,  $t_{fuse2}$ ]
clc; clear; close all;

%% GA
nvars = 8;
lb = [0, 70, 0, 0, 0, 0, 0, 0];
ub = [2*pi, 140, 40, 40, 40, 40, 40, 40];

theta0 = pi;
v_FY1_speed0 = 120;
t_release0 = 0;
t_fuse0 = 2;

populationSize = 200;
initPop = zeros(populationSize, nvars);
initPop(1,:) = [theta0, v_FY1_speed0, t_release0, t_fuse0, t_release0+2, t_fuse0,
    t_release0+4, t_fuse0];

A = [0,0,1,0,-1,0,0,0;
    0,0,0,0,1,0,-1,0];
b = [-1; -1]; %  $t_2 - t_1 \geq 1$ ,  $t_3 - t_2 \geq 1$ 

opts = optimoptions('ga','Display','iter','MaxGenerations',5000,...
    'PopulationSize',populationSize,'InitialPopulationMatrix',initPop);
[x_opt,fval] = ga(@(x) -calcCoverage(x), nvars, A, b, [], [], lb, ub, [], opts);

fprintf('\n最优解:\n');

```

```

fprintf('theta = %.4f rad\n', x_opt(1));
fprintf('v_FY1 = %.4f m/s\n', x_opt(2));
for i = 1:3
    fprintf('烟雾弹%d: t_release = %.4f s, t_fuse = %.4f s\n', i, x_opt(2*i+1), x_opt(2*i+2));
end
fprintf('最大遮蔽时长 = %.4f s\n', -fval);

%% 目标函数
function Tcover = calcCoverage(x)
    theta      = x(1);
    v_FY1_speed = x(2);
    t_release   = [x(3), x(5), x(7)];
    t_fuse      = [x(4), x(6), x(8)];

    FY1_0 = [17800,0,1800];
    M1_0 = [20000,0,2000];
    v_M1 = 300;

    rSmoke = 10;
    g = 9.81;

    cylBot = [0,200,0];
    H = 10; R = 7; cylTop = cylBot + [0,0,H];

    dir_FY1 = [cos(theta), sin(theta), 0];
    v_FY1 = v_FY1_speed * dir_FY1;

    smokes = cell(1,3);
    for i = 1:3
        posRel = FY1_0 + v_FY1 * t_release(i);
        posExp = posRel + v_FY1 * t_fuse(i) + 0.5*[0,0,-g]*t_fuse(i)^2;
        tExp = t_release(i) + t_fuse(i);
        smokes{i} = struct('time', tExp, 'pos', posExp);
    end

    tStart = min(cellfun(@(s) s.time, smokes));
    tEnd = min(max(cellfun(@(s) s.time, smokes)) + 20, 67);

    dir_M1 = -M1_0; dir_M1 = dir_M1/norm(dir_M1);
    v_M1 = v_M1 * dir_M1;

    dt = 0.005; N = floor((tEnd - tStart)/dt);
    covered = false(N,1);

    for n = 1:N
        tCur = tStart + n*dt;
        M1_pos = M1_0 + v_M1 * tCur;
    end
end

```

```

        if cylCovered(M1_pos, cylBot, cylTop, R, smokes, tCur, rSmoke)
            covered(n) = true;
        end
    end

    Tcover = sum(covered)*dt;
end

%% 圆柱体遮蔽判定
function covered = cylCovered(obsPos, cylBot, cylTop, cylR, smokes, tCur, rSmoke)
    persistent samplePts cylBotP cylTopP cylRP
    if isempty(samplePts) || any(cylBot ~= cylBotP) || any(cylTop ~= cylTopP) || cylR ~= cylRP
        hSamp = 40; aSamp = 16;
        [hGrid,aGrid] = meshgrid(0:hSamp, 0:aSamp-1);

        posH = cylBot + (hGrid(:)/hSamp).*(cylTop - cylBot);
        ang = 2*pi*aGrid(:)/aSamp;

        samplePts = [cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];

        cylBotP = cylBot; cylTopP = cylTop; cylRP = cylR;
    end

    for i = 1:size(samplePts,1)
        if ~pointCovered(obsPos, samplePts(i,:), smokes, tCur, rSmoke)
            covered = false;
            return;
        end
    end
    covered = true;
end

%% 单点遮蔽判定
function covered = pointCovered(obsPos, pt, smokes, tCur, rSmoke)
    covered = false;
    for i = 1:numel(smokes)
        tExp = smokes{i}.time;
        if tCur >= tExp && tCur <= tExp + 20
            smokeC = smokes{i}.pos + [0,0,-3]*(tCur - tExp);
            if lineIntersectSphere(obsPos, pt, smokeC, rSmoke)
                covered = true;
                return;
            end
        end
    end
end
end

```

```

%% 线段与球体相交判定
function intersect = lineIntersectSphere(A,B,C,r)
    AB = B - A;
    AC = C - A;
    t = dot(AB,AC)/dot(AB,AB);
    t = max(0,min(1,t));
    closest = A + t*AB;
    intersect = norm(closest - C) <= r;
end

```

最优解:

投弹角度 $\theta = 3.1339$ rad

投弹速度 $v_{FY1} = 121.2417$ m/s

烟雾弹1: 释放时间 = 0.0993 s, 引信时间 = 3.4747 s

烟雾弹2: 释放时间 = 3.1929 s, 引信时间 = 4.8082 s

烟雾弹3: 释放时间 = 5.1680 s, 引信时间 = 5.4649 s

最大遮蔽时长 = 6.8800 s

3.4 问题3 结果分析代码

```

% q3_s.m
clc; clear; close all;

%% 参数设置
theta      = 3.13386577744472;
v_FY1      = 121.241733878003;
t_release  = [0.0993010070499214, 3.19287468517247, 5.16803229288553];
t_fuse     = [3.47468751782884, 4.80818456582672, 5.46485872367104];

FY1_0 = [17800,0,1800];
M1_0 = [20000,0,2000];
v_M1 = 300;
rSmoke = 10;
vSmoke = [0,0,-3];
g      = 9.81;
cylBot = [0,200,0];
H      = 10; R = 7;
cylTop = cylBot + [0,0,H];
maxDur = 20;

%% FY1 飞行方向
dir_FY1 = [cos(theta), sin(theta), 0];
vVec_FY1 = v_FY1 * dir_FY1;

```

```

%% 计算三颗烟雾弹爆炸时间与位置
numBombs = length(t_release);
explosions = cell(1,numBombs);
for i = 1:numBombs
    posRel = FY1_0 + vVec_FY1 * t_release(i);
    tExp = t_release(i) + t_fuse(i);
    posExp = posRel + vVec_FY1*t_fuse(i) + 0.5*[0,0,-g]*t_fuse(i)^2;
    explosions{i} = struct('time', tExp, 'pos', posExp, 'dur', maxDur);
end

%% M1 运动
dir_M1 = -M1_0/norm(M1_0);
vVec_M1 = v_M1 * dir_M1;

%% 仿真与评估
dt = 0.005;
Tcover_ray = zeros(numBombs,1);
Tcover_cone = zeros(numBombs,1);
tVec_bomb = cell(numBombs,1);
cov_ray_bomb = cell(numBombs,1);
cov_cone_bomb = cell(numBombs,1);

for i = 1:numBombs
    exp = explosions{i};
    t0 = exp.time;
    tVec = t0:dt:(t0 + exp.dur);
    N = length(tVec);

    cov_ray = false(N,1);
    cov_cone = false(N,1);
    singleExp = { struct('time',exp.time,'pos',exp.pos,'dur',exp.dur) };

    for n = 1:N
        tCur = tVec(n);
        M1_pos = M1_0 + vVec_M1 * tCur;

        cov_ray(n) = cylCoveredRay(M1_pos, cylBot, cylTop, R, singleExp, tCur, rSmoke);

        smokePos = exp.pos + vSmoke*(tCur - exp.time);
        L = norm(smokePos - M1_pos);
        if L > rSmoke
            u = (smokePos - M1_pos)/L;
            alpha = asin(min(1,rSmoke/L));
            cov_cone(n) = cylInCone(M1_pos, cylBot, cylTop, R, u, alpha);
        else
            cov_cone(n) = true;
        end
    end
end

```

```

end

Tcover_ray(i) = sum(cov_ray)*dt;
Tcover_cone(i) = sum(cov_cone)*dt;
tVec_bomb{i} = tVec;
cov_ray_bomb{i} = cov_ray;
cov_cone_bomb{i} = cov_cone;
end

%% 输出
fprintf('每颗烟雾弹独立评估\n');
for i = 1:numBombs
    fprintf('弹 %d: tExp = %.4f s, pos = [%.4f, %.4f, %.4f]\n', ...
        i, explosions{i}.time, explosions{i}.pos);
    fprintf(' 射线法 遮蔽时长: %.4f s\n', Tcover_ray(i));
    fprintf(' 圆锥法 遮蔽时长: %.4f s\n', Tcover_cone(i));
end

%% 可视化
figure('Name','每颗烟雾弹遮蔽状态','NumberTitle','off','Position',[100 100 1000 600]);
tiledlayout(2,numBombs,'Padding','compact','TileSpacing','compact');
for i = 1:numBombs
    nexttile(i);
    stairs(tVec_bomb{i}, double(cov_ray_bomb{i}),'LineWidth',1.2);
    title(sprintf('弹 %d - 射线法', i)); xlabel('时间(s)'); ylabel('遮蔽'); ylim([-0.1 1.1]);
    grid on;

    nexttile(i+numBombs);
    stairs(tVec_bomb{i}, double(cov_cone_bomb{i}),'LineWidth',1.2);
    title(sprintf('弹 %d - 圆锥法', i)); xlabel('时间(s)'); ylabel('遮蔽'); ylim([-0.1 1.1]);
    grid on;
end

function covered = cylCoveredRay(obsPos, cylBot, cylTop, cylR, explosions, tCur, rSmoke)
persistent sampPts cylBotP cylTopP cylRP
if isempty(sampPts) || any(cylBot ~= cylBotP) || any(cylTop ~= cylTopP) || cylR ~= cylRP
    hSamp = 40; aSamp = 16;
    [aGrid,hGrid] = meshgrid(0:aSamp-1, 0:hSamp);
    ang = 2*pi*aGrid(:)/aSamp;
    heights = hGrid(:)/hSamp;
    posH = cylBot + heights.*(cylTop-cylBot);
    sampPts = [cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];
    cylBotP = cylBot; cylTopP = cylTop; cylRP = cylR;
end
covered = true;
for i = 1:size(sampPts,1)
    if ~pointCovered(obsPos, sampPts(i,:), explosions, tCur, rSmoke)

```

```

        covered = false; return;
    end
end
end

function covered = pointCovered(obsPos, pt, explosions, tCur, rSmoke)
    covered = false;
    for i = 1:numel(explosions)
        tExp = explosions{i}.time;
        dur = explosions{i}.dur;
        if tCur >= tExp && tCur <= tExp+dur
            smokePos = explosions{i}.pos + [0,0,-3]*(tCur-tExp);
            if lineIntersectSphere(obsPos, pt, smokePos, rSmoke)
                covered = true; return;
            end
        end
    end
end

function intersect = lineIntersectSphere(A,B,C,r)
    AB = B - A;
    if all(AB==0)
        intersect = norm(A-C) <= r; return;
    end
    t = dot(AB,C-A)/dot(AB,AB);
    t = max(0,min(1,t));
    closest = A + t*AB;
    intersect = norm(closest - C) <= r;
end

function inside = cylInCone(obsPos, cylBot, cylTop, cylR, u, alpha)
    inside = isDiskInCone(obsPos, cylBot, cylR, u, alpha) && isDiskInCone(obsPos, cylTop, cylR,
        u, alpha);
end

function inside = isDiskInCone(V, C, r, u, alpha)
    tol = 1e-12;
    negTol = -1e-9;

    w = C(:) - V(:);
    s0 = dot(w, u(:));
    Ax = u(1); Ay = u(2);
    A = Ax; B = Ay;
    C1 = w(1); C2 = w(2);
    W2 = dot(w,w);
    k = (cos(alpha))^2;

```

```

amp = r * sqrt(A^2 + B^2);
if s0 - amp < -tol
    inside = false;
    return;
end

% 构造 h(theta) = s(theta)^2 - ||P-V||^2 * cos^2(alpha)
% 化简后得到形式: h = C0 + Cc*cos + Cs*sin + Ccos2*cos2 + Csin2*sin2
C0 = s0^2 - k*(W2 + r^2) + r^2*(A^2 + B^2)/2;
Cc = 2*s0*r*A - 2*k*r*C1;
Cs = 2*s0*r*B - 2*k*r*C2;
Ccos2 = r^2*(A^2 - B^2)/2;
Csin2 = r^2*(A*B);

% 将 h(theta) * (1+t^2)^2 表示为四次多项式 P(t) = c4 t^4 + c3 t^3 + c2 t^2 + c1 t + c0
c0 = C0 + Cc + Ccos2;
c1 = 2*Cs + 4*Csin2;
c2 = 2*C0 - 6*Ccos2;
c3 = 2*Cs - 4*Csin2;
c4 = C0 - Cc + Ccos2;

% 多项式系数向量
p = [c4, c3, c2, c1, c0];

% 若最高阶系数为负, 则多项式在无穷处趋向 -inf, 必有负值 -> 不满足
if c4 < -tol
    inside = false;
    return;
end

% 求导数并找出导数的实根 (候选极值点)
dp = [4*c4, 3*c3, 2*c2, c1];
rts = roots(dp);
cand = [];
for i = 1:numel(rts)
    if abs(imag(rts(i))) < 1e-8
        cand(end+1) = real(rts(i));
    end
end

% 加入 t=0 (对应 theta=0) 的检查点
cand(end+1) = 0;

% 如果最高阶系数接近零, 说明 P 可能降阶, 为保险起见在很大 t 处再检查一次
if abs(c4) < tol
    T = 1e6;
    cand(end+1) = T;
    cand(end+1) = -T;
end

```

```

end

Pvals = polyval(p, cand);
Pmin = min(Pvals);

% 因为  $h(\theta) = P(t) / (1+t^2)^2$ , 分母始终>0, 所以  $P(t)$  在全实轴非负等价于  $h(\theta)$  对所有
%  $\theta$  非负
if Pmin < negTol
    inside = false;
else
    inside = true;
end
end
end
--- 每颗烟雾弹独立评估 ---
弹 1: tExp = 3.5740 s, pos = [17366.6964, 3.3482, 1740.7797]
    射线法 遮蔽时长: 3.4200 s
    圆锥法 遮蔽时长: 3.4200 s
弹 2: tExp = 8.0011 s, pos = [16829.9667, 7.4955, 1686.6031]
    射线法 遮蔽时长: 2.6400 s
    圆锥法 遮蔽时长: 2.6400 s
弹 3: tExp = 10.6329 s, pos = [16510.8883, 9.9610, 1653.5137]
    射线法 遮蔽时长: 1.0600 s
    圆锥法 遮蔽时长: 1.0600 s

```

3.5 问题4 仿真与优化代码

```

% q4.m
clc; clear; close all;
%% GA
numVars = 12; % [theta1,v1,tR1,tF1, theta2,v2,tR2,tF2, theta3,v3,tR3,tF3]
lb = [0,70,0,0, 0,70,0,0, 0,70,0,0];
ub = [2*pi,140,40,40, 2*pi,140,40,40, 2*pi,140,40,40];
theta0 = pi;
theta1 = 4.0843;
theta2 = 2.0216;
v0 = 100;
v1 = 125;
v2 = 125;
tR0 = 0.8;
tF0 = 3.3;
tR1 = 6.5;
tF1 = 7.3;
tR2 = 21;
tF2 = 7;

```

```

popSize = 200;
initPop = zeros(popSize,numVars);
initPop(1,:) = [theta0,v0,tR0,tF0, theta1,v1,tR1,tF1, theta2,v2,tR2,tF2];
gaOpts = optimoptions('ga', 'Display','iter', 'MaxGenerations',2000, ...
    'PopulationSize',popSize, 'InitialPopulationMatrix',initPop);

[optSol, optVal] = ga(@(x) -multiSmokeCovTime(x), numVars, [], [], [], [], lb, ub, [], gaOpts);

fprintf('\n最优解:\n');
for uav = 1:3
    fprintf('无人机%d:   = %.4f rad, v = %.4f m/s, 释放 = %.4f s, 引信 = %.4f s\n', ...
        uav, optSol(4*(uav-1)+1), optSol(4*(uav-1)+2), ...
        optSol(4*(uav-1)+3), optSol(4*(uav-1)+4));
end
fprintf('最大遮蔽时长 = %.4f s\n', -optVal);

%% 目标函数
function covTime = multiSmokeCovTime(vars)
    posUAV = {[17800,0,1800], [12000,1400,1400], [6000,-3000,700]};
    g = 9.81; rSmoke = 10;

    cylBot = [0,200,0];
    cylH = 10; cylR = 7;
    cylTop = cylBot + [0,0,cylH];

    explosions = cell(1,3);
    for uav = 1:3
        theta = vars(4*(uav-1)+1);
        v      = vars(4*(uav-1)+2);
        tR      = vars(4*(uav-1)+3);
        tF      = vars(4*(uav-1)+4);

        dir = [cos(theta), sin(theta), 0];
        vVec = v*dir;
        relPos = posUAV{uav} + vVec*tR;
        expPos = relPos + vVec*tF + 0.5*[0,0,-g]*tF^2;
        expTime = tR + tF;

        explosions{uav} = struct('time',expTime,'pos',expPos);
    end

    % M1运动
    posM1 = [20000,0,2000];
    vM1 = 300;
    dirM1 = -posM1/norm(posM1);
    vVecM1 = vM1*dirM1;

```

```

% 仿真时间
tStart = min(cellfun(@(s) s.time, explosions));
tEnd = min(max(cellfun(@(s) s.time, explosions))+20, 67);
dt = 0.005;
Nsteps = floor((tEnd-tStart)/dt);
covered = false(Nsteps,1);

for i = 1:Nsteps
    tCur = tStart + i*dt;
    posCur = posM1 + vVecM1*tCur;
    if cylCovered(posCur, cylBot, cylTop, cylR, explosions, tCur, rSmoke)
        covered(i) = true;
    end
end

covTime = sum(covered)*dt;
end

%% 圆柱体遮蔽判定
function covered = cylCovered(obsPos, cylBot, cylTop, cylR, explosions, tCur, rSmoke)
persistent sampPts cylBotP cylTopP cylRP
if isempty(sampPts) || any(cylBot~=cylBotP) || any(cylTop~=cylTopP) || cylR~=cylRP
    nH = 2; nA = 4;
    [hG,aG] = meshgrid(0:nH,0:nA-1);
    heights = hG(:)/nH;
    posH = cylBot + heights.*(cylTop-cylBot);
    ang = 2*pi*aG(:)/nA;
    sampPts = [cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];
    cylBotP = cylBot; cylTopP = cylTop; cylRP = cylR;
end
covered = true;
for i = 1:size(sampPts,1)
    if ~pointCovered(obsPos, sampPts(i,:), explosions, tCur, rSmoke)
        covered = false; return;
    end
end
end

function covered = pointCovered(obsPos, pt, explosions, tCur, rSmoke)
covered = false;
for i = 1:numel(explosions)
    tExp = explosions{i}.time;
    if tCur>=tExp && tCur<=tExp+20
        smokePos = explosions{i}.pos + [0,0,-3]*(tCur-tExp);
        if lineIntersectSphere(obsPos, pt, smokePos, rSmoke)
            covered = true; return;
        end
    end
end

```

```

        end
    end
end

function intersect = lineIntersectSphere(A,B,C,r)
    AB = B-A;
    AC = C-A;
    t = dot(AB,AC)/dot(AB,AB);
    t = max(0,min(1,t));
    closest = A + t*AB;
    intersect = norm(closest-C) <= r;
end

```

最优解:

无人机1: = 3.1221 rad, $v = 101.1553$ m/s, 释放 = 0.8000 s, 引信 = 3.3055 s
 无人机2: = 4.0843 rad, $v = 123.8281$ m/s, 释放 = 6.5000 s, 引信 = 7.3000 s
 无人机3: = 2.0231 rad, $v = 123.7188$ m/s, 释放 = 21.0000 s, 引信 = 7.0175 s
 最大遮蔽时长 = 10.65 s

3.6 问题4 结果分析代码

```

% q4_s.m
clc; clear; close all;

%% 参数设置
theta = [3.1221, 4.0843, 2.0231];
v_FY = [101.1553, 123.8281, 123.7188];
t_release = [0.8000, 6.5000, 21.0000];
t_fuse = [3.3055, 7.3000, 7.0175];
FY0 = { [17800,0,1800], [12000,1400,1400], [6000,-3000,700] };
M1_0 = [20000,0,2000];
v_M1 = 300;
rSmoke = 10;
vSmoke = [0,0,-3];
g = 9.81;
maxDur = 20;
cylBot = [0,200,0];
H = 10; R = 7;
cylTop = cylBot + [0,0,H];

%% 烟幕弹爆炸信息
numBombs = 3;
explosions = cell(1,numBombs);
for i = 1:numBombs
    dir_FY = [cos(theta(i)), sin(theta(i)), 0];

```

```

    vVec_FY = v_FY(i) * dir_FY;
    posRel = FY0{i} + vVec_FY * t_release(i);
    tExp = t_release(i) + t_fuse(i);
    posExp = posRel + vVec_FY * t_fuse(i) + 0.5*[0,0,-g]*t_fuse(i)^2;
    explosions{i} = struct('time', tExp, 'pos', posExp, 'dur', maxDur);
end

%% M1 运动
dir_M1 = -M1_0 / norm(M1_0);
vVec_M1 = v_M1 * dir_M1;

%% 仿真与评估
dt = 0.005;
Tcover_ray = zeros(numBombs,1);
Tcover_cone = zeros(numBombs,1);
tVec_bomb = cell(numBombs,1);
cov_ray_bomb = cell(numBombs,1);
cov_cone_bomb = cell(numBombs,1);

for i = 1:numBombs
    exp = explosions{i};
    t0 = exp.time;
    tVec = t0:dt:(t0+exp.dur);
    N = length(tVec);

    cov_ray = false(N,1);
    cov_cone = false(N,1);
    singleExp = { struct('time',exp.time,'pos',exp.pos,'dur',exp.dur) };

    for n = 1:N
        tCur = tVec(n);
        M1_pos = M1_0 + vVec_M1 * tCur;

        % 射线法
        cov_ray(n) = cylCoveredRay(M1_pos, cylBot, cylTop, R, singleExp, tCur, rSmoke);

        % 圆锥法
        smokePos = exp.pos + vSmoke*(tCur - exp.time);
        L = norm(smokePos - M1_pos);
        if L > rSmoke
            u = (smokePos - M1_pos)/L;
            alpha = asin(min(1,rSmoke/L));
            cov_cone(n) = cylInCone(M1_pos, cylBot, cylTop, R, u, alpha);
        else
            cov_cone(n) = true;
        end
    end
end
end

```

```

Tcover_ray(i) = sum(cov_ray)*dt;
Tcover_cone(i) = sum(cov_cone)*dt;
tVec_bomb{i} = tVec;
cov_ray_bomb{i} = cov_ray;
cov_cone_bomb{i} = cov_cone;
end

%% 输出
for i = 1:numBombs
    fprintf('弹 %d: tExp = %.4f s, pos = [%.4f, %.4f, %.4f]\n', ...
        i, explosions{i}.time, explosions{i}.pos);
    fprintf(' 射线法 遮蔽时长: %.4f s\n', Tcover_ray(i));
    fprintf(' 圆锥法 遮蔽时长: %.4f s\n', Tcover_cone(i));
end

%% 可视化
figure('Name','三架无人机各投一弹 - 遮蔽状态','NumberTitle','off','Position',[100 100 1000 600]);
tiledlayout(2,numBombs,'Padding','compact','TileSpacing','compact');
for i = 1:numBombs
    nexttile(i);
    stairs(tVec_bomb{i}, double(cov_ray_bomb{i}), 'LineWidth',1.2);
    title(sprintf('弹 %d - 射线法', i));
    xlabel('时间(s)'); ylabel('遮蔽'); ylim([-0.1 1.1]); grid on;

    nexttile(i+numBombs);
    stairs(tVec_bomb{i}, double(cov_cone_bomb{i}), 'LineWidth',1.2);
    title(sprintf('弹 %d - 圆锥法', i));
    xlabel('时间(s)'); ylabel('遮蔽'); ylim([-0.1 1.1]); grid on;
end

%% ===== 函数部分 =====
function covered = cylCoveredRay(obsPos, cylBot, cylTop, cylR, explosions, tCur, rSmoke)
persistent sampPts cylBotP cylTopP cylRP
if isempty(sampPts) || any(cylBot ~= cylBotP) || any(cylTop ~= cylTopP) || cylR ~= cylRP
    hSamp = 40; aSamp = 16;
    [aGrid,hGrid] = meshgrid(0:aSamp-1, 0:hSamp);
    ang = 2*pi*aGrid(:)/aSamp;
    heights = hGrid(:)/hSamp;
    posH = cylBot + heights.*(cylTop-cylBot);
    sampPts = [cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];
    cylBotP = cylBot; cylTopP = cylTop; cylRP = cylR;
end
covered = true;
for i = 1:size(sampPts,1)
    if ~pointCovered(obsPos, sampPts(i,:), explosions, tCur, rSmoke)
        covered = false; return;
    end
end

```

```

        end
    end
end

function covered = pointCovered(obsPos, pt, explosions, tCur, rSmoke)
    covered = false;
    for i = 1:numel(explosions)
        tExp = explosions{i}.time;
        dur = explosions{i}.dur;
        if tCur >= tExp && tCur <= tExp+dur
            smokePos = explosions{i}.pos + [0,0,-3]*(tCur-tExp);
            if lineIntersectSphere(obsPos, pt, smokePos, rSmoke)
                covered = true; return;
            end
        end
    end
end

function intersect = lineIntersectSphere(A,B,C,r)
    AB = B - A;
    if all(AB==0)
        intersect = norm(A-C) <= r; return;
    end
    t = dot(AB,C-A)/dot(AB,AB);
    t = max(0,min(1,t));
    closest = A + t*AB;
    intersect = norm(closest - C) <= r;
end

function inside = cylInCone(obsPos, cylBot, cylTop, cylR, u, alpha)
    inside = isDiskInCone(obsPos, cylBot, cylR, u, alpha) && isDiskInCone(obsPos, cylTop, cylR,
        u, alpha);
end

function inside = isDiskInCone(V, C, r, u, alpha)
    tol = 1e-12;
    negTol = -1e-9;

    w = C(:) - V(:);
    s0 = dot(w, u(:));
    Ax = u(1); Ay = u(2);
    A = Ax; B = Ay;
    C1 = w(1); C2 = w(2);
    W2 = dot(w,w);
    k = (cos(alpha))^2;

    amp = r * sqrt(A^2 + B^2);

```

```

if s0 - amp < -tol
    inside = false;
    return;
end

% 构造 h(theta) = s(theta)^2 - ||P-V||^2 * cos^2(alpha)
% 化简后得到形式: h = C0 + Cc*cos + Cs*sin + Ccos2*cos2 + Csin2*sin2
C0 = s0^2 - k*(W2 + r^2) + r^2*(A^2 + B^2)/2;
Cc = 2*s0*r*A - 2*k*r*C1;
Cs = 2*s0*r*B - 2*k*r*C2;
Ccos2 = r^2*(A^2 - B^2)/2;
Csin2 = r^2*(A*B);

% 将 h(theta) * (1+t^2)^2 表示为四次多项式 P(t) = c4 t^4 + c3 t^3 + c2 t^2 + c1 t + c0
c0 = C0 + Cc + Ccos2;
c1 = 2*Cs + 4*Csin2;
c2 = 2*C0 - 6*Ccos2;
c3 = 2*Cs - 4*Csin2;
c4 = C0 - Cc + Ccos2;

% 多项式系数向量
p = [c4, c3, c2, c1, c0];

% 若最高阶系数为负, 则多项式在无穷处趋向 -inf, 必有负值 -> 不满足
if c4 < -tol
    inside = false;
    return;
end

% 求导数并找出导数的实根 (候选极值点)
dp = [4*c4, 3*c3, 2*c2, c1];
rts = roots(dp);
cand = [];
for i = 1:numel(rts)
    if abs(imag(rts(i))) < 1e-8
        cand(end+1) = real(rts(i));
    end
end

% 加入 t=0 (对应 theta=0) 的检查点
cand(end+1) = 0;

% 如果最高阶系数接近零, 说明 P 可能降阶, 为保险起见在很大 t 处再检查一次
if abs(c4) < tol
    T = 1e6;
    cand(end+1) = T;
    cand(end+1) = -T;
end

```

```

Pvals = polyval(p, cand);
Pmin = min(Pvals);

% 因为  $h(\theta) = P(t) / (1+t^2)^2$ , 分母始终>0, 所以  $P(t)$  在全实轴非负等价于  $h(\theta)$  对所有
%  $\theta$  非负
if Pmin < negTol
    inside = false;
else
    inside = true;
end
end

弹 1: 爆炸时刻 = 4.1055 s, 爆炸位置 = [17384.7858, 8.0947, 1746.4064]
    射线相交法 遮蔽时长: 4.4700 s
    圆锥覆盖法 遮蔽时长: 4.4700 s
弹 2: 爆炸时刻 = 13.8000 s, 爆炸位置 = [10995.8936, 17.2988, 1138.6126]
    射线相交法 遮蔽时长: 3.4500 s
    圆锥覆盖法 遮蔽时长: 3.4500 s
弹 3: 爆炸时刻 = 28.0175 s, 爆炸位置 = [4485.0964, 117.7306, 458.4518]
    射线相交法 遮蔽时长: 2.7300 s
    圆锥覆盖法 遮蔽时长: 2.7300 s

```

3.7 问题5 仿真与优化代码

```

% q5.m
clc; clear; close all;

%% GA
nUAV = 5;
nBomb = 3;
varsPerUAV = 8;
nvars = nUAV * varsPerUAV;
% 8 个变量: [theta, v, t_release1, t_fuse1, t_release2, t_fuse2, t_release3, t_fuse3]
initGuess = [
    3.1339, 121, 0.1, 3.5, 3.1, 4.5, 5.1, 5.5;
    3.8574, 120.8979, 2.8133, 6.00, 6.4717, 6.25, 8.8673, 8.28;
    1.7040, 120.5277, 19.9799, 4.21, 23.4986, 3.78, 28.8131, 2.64;
    3.8913, 114.6142, 8.6996, 12.5674, 15.6559, 13.4961, 16.7156, 6.67;
    2.1148, 120.5786, 13.7658, 1.8331, 14.7926, 4.8365, 20.9975, 1.9457;
];
x0 = reshape(initGuess.', 1, []);

lb = repmat([0, 70, 0, 0, 0, 0, 0, 0], 1, nUAV);

```

```

ub = repmat([2*pi, 140, 40, 40, 40, 40, 40, 40], 1, nUAV);

%% 初始种群
populationSize = 100;
initPop = zeros(populationSize, nvars);
initPop(1,:) = x0;

%% 约束矩阵
A = zeros(nUAV*2, nvars);
b = -ones(nUAV*2, 1);

for u = 0:nUAV-1
    base = u*varsPerUAV;
    row1 = u*2+1;
    A(row1, base+3) = 1; % t1
    A(row1, base+5) = -1; % t2

    row2 = u*2+2;
    A(row2, base+5) = 1; % t2
    A(row2, base+7) = -1; % t3
end

%% GA 求解
opts = optimoptions('ga','Display','iter',...
    'MaxGenerations',2000,'PopulationSize',populationSize,...
    'InitialPopulationMatrix',initPop);

[x_opt, fval] = ga(@(x) -multiUAVCoverage(x), nvars, A, b, [], [], lb, ub, [], opts);

fprintf('\n最优解:\n');
for u=1:nUAV
    idx = (u-1)*varsPerUAV;
    fprintf('UAV%d: theta=%.4f rad, v=%.4f m/s\n', u, x_opt(idx+1), x_opt(idx+2));
    for k=1:nBomb
        fprintf(' 烟雾弹%d: 释放=%.4f s, 引信=%.4f s\n', k, x_opt(idx+2*k+1), x_opt(idx+2*k+2));
    end
end
fprintf('三导弹遮蔽总时长 = %.4f s\n', -fval);

%% 目标函数
function TcoverTotal = multiUAVCoverage(x)
    FY0 = [17800,0,1800;
        12000,1400,1400;
        6000,-3000,700;
        11000,2000,1800;
        13000,-2000,1300];

```

```

M0 = [20000,0,2000;
      19000,600,2100;
      18000,-600,1900];
vM = 300;

cylBot = [0,200,0]; H=10; R=7; cylTop = cylBot+[0,0,H];
rSmoke = 10; g=9.81;

nUAV=5; nBomb=3; varsPerUAV=8;
smokes = {};

for u=1:nUAV
    base = (u-1)*varsPerUAV;
    theta = x(base+1); v_speed = x(base+2);
    dir = [cos(theta),sin(theta),0];
    vUAV = v_speed * dir;
    pos0 = FY0(u,:);

    for k=1:nBomb
        t_release = x(base+2*k+1);
        t_fuse = x(base+2*k+2);
        posRel = pos0 + vUAV*t_release;
        posExp = posRel + vUAV*t_fuse + 0.5*[0,0,-g]*t_fuse^2;
        tExp = t_release+t_fuse;
        smokes{end+1} = struct('time',tExp,'pos',posExp);
    end
end

TcoverTotal = 0;
for m=1:3
    dirM = -M0(m,:); dirM=dirM/norm(dirM);
    vMvec = vM*dirM;

    tStart = min(cellfun(@(s) s.time, smokes));
    tEnd = min(max(cellfun(@(s) s.time, smokes))+20,67);
    dt=0.005; N=floor((tEnd-tStart)/dt);
    covered=false(N,1);

    for n=1:N
        tCur = tStart+n*dt;
        Mpos = M0(m,:) + vMvec*tCur;
        if cylCovered(Mpos,cylBot,cylTop,R,smokes,tCur,rSmoke)
            covered(n)=true;
        end
    end

    TcoverTotal = TcoverTotal + sum(covered)*dt;
end

```

```

    end
end

%% 圆柱体遮蔽判定
function covered = cylCovered(obsPos,cylBot,cylTop,cylR,smokes,tCur,rSmoke)
    persistent sampPts cylBotP cylTopP cylRP
    if isempty(sampPts) || any(cylBot~=cylBotP) || any(cylTop~=cylTopP) || cylR~=cylRP
        hSamp=40; aSamp=16;
        [hGrid,aGrid]=meshgrid(0:hSamp,0:aSamp-1);
        posH=cylBot+(hGrid(:)/hSamp).*(cylTop-cylBot);
        ang=2*pi*aGrid(:)/aSamp;
        sampPts=[cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];
        cylBotP=cylBot; cylTopP=cylTop; cylRP=cylR;
    end
    for i=1:size(sampPts,1)
        if ~pointCovered(obsPos,sampPts(i,:),smokes,tCur,rSmoke)
            covered=false; return;
        end
    end
    covered=true;
end

function covered = pointCovered(obsPos,pt,smokes,tCur,rSmoke)
    covered=false;
    for i=1:numel(smokes)
        tExp=smokes{i}.time;
        if tCur>=tExp && tCur<=tExp+20
            smokeC=smokes{i}.pos+[0,0,-3]*(tCur-tExp);
            if lineIntersectSphere(obsPos,pt,smokeC,rSmoke)
                covered=true; return;
            end
        end
    end
end

function intersect = lineIntersectSphere(A,B,C,r)
    AB=B-A; AC=C-A;
    t=dot(AB,AC)/dot(AB,AB); t=max(0,min(1,t));
    closest=A+t*AB;
    intersect=norm(closest-C)<=r;
end

```

3.8 问题 5 结果分析代码

```
%q5_s.m
```

```

clc; clear; close all;

%% 参数设置
nUAV = 5; nBombPerUAV = 3; nBombs = nUAV * nBombPerUAV; nMissiles = 3;
thetas = [3.1341, 3.8577, 1.7372, 3.8911, 2.1149];
vs      = [121.5454, 121.1187, 120.5351, 114.6156, 120.5811];
t_releases = [
    0.1048, 3.1528, 5.1522;
    2.7448, 6.4729, 8.9347;
    19.9909, 23.5031, 28.9392;
    8.7066, 15.6456, 16.7185;
    13.7684, 14.7942, 21.0690
];
t_fuses = [
    3.4897, 4.5851, 5.5055;
    6.0025, 6.2674, 8.2847;
    4.2478, 3.7896, 1.7055;
    12.5703, 13.5010, 6.6965;
    1.8218, 4.8601, 1.9302
];
FY0 = [17800, 0, 1800;
    12000, 1400, 1400;
    6000, -3000, 700;
    11000, 2000, 1800;
    13000, -2000, 1300];
M0 = [20000, 0, 2000;
    19000, 600, 2100;
    18000, -600, 1900];
vMissile = 300;
rSmoke = 10;
vSmoke = [0,0,-3];
g = 9.81;
cylBot = [0,200,0];
H = 10; R = 7;
cylTop = cylBot + [0,0,H];
maxDur = 20;

%% 烟幕弹爆炸信息
explosions = cell(nBombs,1);
idx = 0;
for u = 1:nUAV
    dir_u = [cos(thetas(u)), sin(thetas(u)), 0];
    vVec_u = vs(u) * dir_u;
    for b = 1:nBombPerUAV
        idx = idx + 1;
        tr = t_releases(u,b);
        tf = t_fuses(u,b);
    end
end

```

```

    posRel = FY0(u,:) + vVec_u * tr;
    tExp = tr + tf;
    posExp = posRel + vVec_u*tf + 0.5*[0,0,-g]*tf^2;
    explosions{idx} = struct('time', tExp, 'pos', posExp, 'dur', maxDur, ...
                             'uav', u, 'bombInUAV', b);

    end
end

%% 仿真与评估
Tcover_ray = zeros(nBombs,nMissiles);
Tcover_cone = zeros(nBombs,nMissiles);
tVec_bomb = cell(nBombs,1);
cov_ray_bomb = cell(nBombs,1);
cov_cone_bomb = cell(nBombs,1);

dt = 0.005;

% 导弹速度向量
vMissVecs = zeros(nMissiles,3);
for m = 1:nMissiles
    dirM = -MO(m,:) / norm(MO(m,:));
    vMissVecs(m,:) = vMissile * dirM;
end

for i = 1:nBombs
    exp = explosions{i};
    t0 = exp.time;
    tVec = t0:dt:(t0 + exp.dur);
    N = numel(tVec);

    cov_ray_all = false(N,nMissiles);
    cov_cone_all = false(N,nMissiles);

    for n = 1:N
        tCur = tVec(n);
        smokePos = exp.pos + vSmoke*(tCur - exp.time);

        for m = 1:nMissiles
            Mpos = MO(m,:) + vMissVecs(m,:) * tCur;

            cov_ray_all(n,m) = cylCoveredRay(Mpos, cylBot, cylTop, R, ...
                {struct('time',exp.time,'pos',exp.pos,'dur',exp.dur)}, tCur, rSmoke);

            L = norm(smokePos - Mpos);
            if L > rSmoke
                u = (smokePos - Mpos)/L;
                alpha = asin(min(1,rSmoke/L));
            end
        end
    end
end

```

```

        cov_cone_all(n,m) = cylInCone(Mpos, cylBot, cylTop, R, u, alpha);
    else
        cov_cone_all(n,m) = true;
    end
end
end

for m = 1:nMissiles
    Tcover_ray(i,m) = sum(cov_ray_all(:,m))*dt;
    Tcover_cone(i,m) = sum(cov_cone_all(:,m))*dt;
end

tVec_bomb{i} = tVec;
cov_ray_bomb{i} = cov_ray_all;
cov_cone_bomb{i} = cov_cone_all;
end

%% 输出
for i = 1:nBombs
    exp = explosions{i};
    fprintf('\n烟幕弹 %2d (UAV %d - 第 %d 枚):\n', i, exp.uav, exp.bombInUAV);
    fprintf('  引爆时刻 = %.4f s, 位置 = [%.4f, %.4f, %.4f]\n', ...
        exp.time, exp.pos(1), exp.pos(2), exp.pos(3));
    for m = 1:nMissiles
        fprintf('    导弹 M%d: 射线法 = %.4f s, 圆锥法 = %.4f s\n', ...
            m, Tcover_ray(i,m), Tcover_cone(i,m));
    end
    fprintf(' >>> 总和: 射线法 = %.4f s, 圆锥法 = %.4f s\n', ...
        sum(Tcover_ray(i,:)), sum(Tcover_cone(i,:)));
end

%% 图像展示
figure('Name','烟幕弹-导弹遮蔽矩阵','NumberTitle','off');
subplot(1,2,1); imagesc(Tcover_ray); colorbar;
title('射线法 (s)'); xlabel('导弹编号'); ylabel('烟幕弹编号');
subplot(1,2,2); imagesc(Tcover_cone); colorbar;
title('圆锥法 (s)'); xlabel('导弹编号'); ylabel('烟幕弹编号');

function covered = cylCoveredRay(obsPos, cylBot, cylTop, cylR, explosions, tCur, rSmoke)
persistent sampPts cylBotP cylTopP cylRP
if isempty(sampPts) || any(cylBot ~= cylBotP) || any(cylTop ~= cylTopP) || cylR ~= cylRP
    hSamp = 40; aSamp = 16;
    [aGrid,hGrid] = meshgrid(0:aSamp-1,0:hSamp);
    ang = 2*pi*aGrid(:)/aSamp;
    heights = hGrid(:)/hSamp;
    posH = cylBot + heights.*(cylTop-cylBot);
    sampPts = [cylR*cos(ang)+posH(:,1), cylR*sin(ang)+posH(:,2), posH(:,3)];
end

```

```

        cylBotP = cylBot; cylTopP = cylTop; cylRP = cylR;
    end
    covered = true;
    for i = 1:size(sampPts,1)
        if ~pointCovered(obsPos,sampPts(i,:),explosions,tCur,rSmoke)
            covered = false; return;
        end
    end
end

function covered = pointCovered(obsPos,pt,explosions,tCur,rSmoke)
    covered = false;
    for i = 1:numel(explosions)
        tExp = explosions{i}.time; dur = explosions{i}.dur;
        if tCur >= tExp && tCur <= tExp+dur
            smokePos = explosions{i}.pos + [0,0,-3]*(tCur-tExp);
            if lineIntersectSphere(obsPos,pt,smokePos,rSmoke)
                covered = true; return;
            end
        end
    end
end

function intersect = lineIntersectSphere(A,B,C,r)
    AB = B-A;
    if all(AB==0), intersect = norm(A-C)<=r; return; end
    t = dot(AB,C-A)/dot(AB,AB); t = max(0,min(1,t));
    closest = A + t*AB;
    intersect = norm(closest-C)<=r;
end

function inside = cylInCone(obsPos,cylBot,cylTop,cylR,u,alpha)
    inside = isDiskInCone(obsPos,cylBot,cylR,u,alpha) && ...
        isDiskInCone(obsPos,cylTop,cylR,u,alpha);
end

function inside = isDiskInCone(V, C, r, u, alpha)
    tol = 1e-12;
    negTol = -1e-9;

    w = C(:) - V(:);
    s0 = dot(w, u(:));
    Ax = u(1); Ay = u(2);
    A = Ax; B = Ay;
    C1 = w(1); C2 = w(2);
    W2 = dot(w,w);
    k = (cos(alpha))^2;

```

```

amp = r * sqrt(A^2 + B^2);
if s0 - amp < -tol
    inside = false;
    return;
end

% 构造 h(theta) = s(theta)^2 - ||P-V||^2 * cos^2(alpha)
% 化简后得到形式: h = C0 + Cc*cos + Cs*sin + Ccos2*cos2 + Csin2*sin2
C0 = s0^2 - k*(W2 + r^2) + r^2*(A^2 + B^2)/2;
Cc = 2*s0*r*A - 2*k*r*C1;
Cs = 2*s0*r*B - 2*k*r*C2;
Ccos2 = r^2*(A^2 - B^2)/2;
Csin2 = r^2*(A*B);

% 将 h(theta) * (1+t^2)^2 表示为四次多项式 P(t) = c4 t^4 + c3 t^3 + c2 t^2 + c1 t + c0
c0 = C0 + Cc + Ccos2;
c1 = 2*Cs + 4*Csin2;
c2 = 2*C0 - 6*Ccos2;
c3 = 2*Cs - 4*Csin2;
c4 = C0 - Cc + Ccos2;

% 多项式系数向量
p = [c4, c3, c2, c1, c0];

% 若最高阶系数为负, 则多项式在无穷处趋向 -inf, 必有负值 -> 不满足
if c4 < -tol
    inside = false;
    return;
end

% 求导数并找出导数的实根 (候选极值点)
dp = [4*c4, 3*c3, 2*c2, c1];
rts = roots(dp);
cand = [];
for i = 1:numel(rts)
    if abs(imag(rts(i))) < 1e-8
        cand(end+1) = real(rts(i));
    end
end

% 加入 t=0 (对应 theta=0) 的检查点
cand(end+1) = 0;

% 如果最高阶系数接近零, 说明 P 可能降阶, 为保险起见在很大 t 处再检查一次
if abs(c4) < tol
    T = 1e6;
    cand(end+1) = T;
end

```

```

        cand(end+1) = -T;
    end

    Pvals = polyval(p, cand);
    Pmin = min(Pvals);

    % 因为  $h(\theta) = P(t) / (1+t^2)^2$ , 分母始终>0, 所以  $P(t)$  在全实轴非负等价于  $h(\theta)$  对所有
    %  $\theta$  非负
    if Pmin < negTol
        inside = false;
    else
        inside = true;
    end
end

```